

Have a thing? Reasoning around recursion with dynamic typing in grounded arithmetic

Preliminary draft – not for distribution

Elliot Bobrow and Bryan Ford and Sascha Kehrli and Stefan Milenković

July 12, 2025

Abstract

Neither the classical nor intuitionistic logic traditions are perfectly-aligned with the purpose of reasoning about computation, in that neither logical tradition can normally permit the direct expression of arbitrary general-recursive functions without inconsistency. We introduce *grounded arithmetic* or GA, a minimalistic but nonetheless powerful foundation for formal reasoning that allows the direct expression of arbitrary recursive definitions. GA adjusts the traditional inference rules such that terms that express nonterminating computations harmlessly denote no semantic value (i.e., $\perp$) instead of leading into logical paradox or inconsistency. Recursive functions may be proven terminating in GA essentially by “dynamically typing” terms, or equivalently, symbolically reverse-executing the computations they denote via GA’s inference rules. Once recursive functions have been proven terminating, logical reasoning about their results reduce to the familiar classical rules. A mechanically-checked consistency proof in Isabelle/HOL exists for the basic quantifier-free fragment of GA. Quantifiers may be added atop this foundation as ordinary computations, whose inference rules are thus admissible and do not introduce new inconsistency risks. While GA is only a first step towards richly-typed grounded deduction practical for everyday use in manual or automated computational reasoning, it shows the promise that the expressive freedom of arbitrary recursive definition can in principle be incorporated into formal systems.

1 Introduction

Today’s standard practices for reasoning about computations, both in traditional “pencil-and-paper” proofs and for automated reasoning and mechanical verification purposes, ultimately derive from either the classical or intuitionistic logic traditions. Classical logic offers the strongest and most familiar deduction rules, including the law of excluded middle (LEM) and proof by contradiction. The intuitionistic tradition rejects LEM and proof by contradiction, but is nevertheless appealing for computational reasoning purposes because an intuitionistic proof that an object with certain properties exists yields in principle a concrete algorithm to construct such an object.

In both logic traditions, one must in general carefully justify all recursive definitions intended to represent computations, by proving their termination, before using a recursive definition. A fundamental part of the expressive power of Turing-complete computation, however, is that computations need not always terminate, and sometimes intentionally do not, as in streaming processes. Allowing direct expression of non-terminating recursive definitions in classical or intuitionistic formal-reasoning systems leads to inconsistency via logical

paradoxes such as the Liar or Curry’s paradox. Despite being common and natural in modern casual programming practice, therefore, unrestricted (potentially non-terminating) recursion remains a problematic and “foreign” challenge in formal reasoning.

Could we bring the “freedom of expression” that we enjoy in casual programming – to employ recursive definitions without restriction – into the world of consistent formal reasoning? This paper introduces *grounded deduction*, an approach inspired by Kripke’s theory of truth [Kripke, 1975] and the body of experimental paraconsistent logics derived from it [Field, 2008]. While paraconsistent logics are not new, the conventional wisdom is that their rules are too weak and/or strange to be usable in practical formal reasoning. Paraconsistent logics have to this point largely served only as intellectual “toys” for metalogical study, and not practical tools usable in real proofs, whether pen-and-paper or automated.

This paper’s main contribution is to take a first step toward making grounded or paraconsistent reasoning usable and practical, in the context of a minimalistic but nevertheless powerful system we call *grounded arithmetic* or GA, for paraconsistent reasoning about natural numbers. GA unsurprisingly parallels classical formal systems of arithmetic such as Peano arithmetic (PA) and Skolem’s primitive-recursive arithmetic (PRA) [Mendelson, 2015, Skolem, 1923], while adjusting the logical deduction rules for negation, implication, and quantification to tolerate unrestricted recursive definitions without inconsistency. Following in the tradition of PA and PRA, intuitionistic Heyting Arithmetic (HA) [Heyting, 1971], and minimalist programming formalisms like LCF and PCF [Plotkin, 1977], we focus particularly on *arithmetic* at present to keep the target domain of discourse conceptually simple (only the natural numbers), while embodying enough power to express full Turing-complete computation capability. Unlike a “pure” paraconsistent logic studied independently of any particular domain of discourse, therefore, GA is plausibly powerful enough for use in practical reasoning about arbitrary computations.

A key insight of this work is that we can view the necessary modifications to the logical deduction rules as implementing an arguably-natural principle we call *habeas quid*: one must first “have a thing” in order to use it in further reasoning. Following this principle we add what may be reasonably viewed as *dynamic typing* preconditions to certain basic inference rules: “dynamic” not in a time-varying sense, as formal arithmetic has no notion of time, but rather in the sense that typing depends on what a term *actually computes* from given inputs, rather than depending on static syntactic structure. GA’s *habeas quid* or dynamic-typing rules avoid inconsistency in the presence of unconstrained recursion by converting logical paradoxes into harmless circular proof obligations. For example, to prove that the Liar paradox ‘ $L \equiv \neg L$ ’ denotes “a thing” (i.e., *habeas quid*, or “is dynamically well-typed”), one would *first* have to prove that L already denotes a thing. In a sense, grounded deduction takes dependent typing [Martin-Löf, 1972, Norell, 2007] to a logical extreme by deferring termination-proof obligations to as “late” as apparently possible. Proving a term dynamically well-typed in GA, i.e., that it terminates with a value, also amounts in essence to executing it symbolically in reverse, as we will see.

A second key insight is that the grounded rules for quantification, and even for mathematical induction, need not be primitives of the target logic, but may instead be considered metalogical shorthands for ordinary computations already expressible in a simpler *basic grounded arithmetic* or BGA. That is, unlike in classical first-order Peano arithmetic (PA) for example, GA’s universal and existential quantifiers are fully computable. Like classical logic but unlike intuitionistic logic, GA’s universal and existential quantifiers remain duals of each other: e.g., the equivalence ‘ $\forall x p(x) \equiv \neg \exists x \neg p(x)$ ’ continues to hold despite other important differences in the quantification rules.

This interpretation of GA’s quantifiers as shorthands for computations relies on essentially the same formal reflection techniques introduced by Gödel in his famous incompleteness theorems. Gödel’s first incompleteness theorem itself – that a consistent (classical) formal system must be syntactically incomplete, or incapable of proving or disproving all well-formed formulas – remains applicable to GA, but becomes nearly trivial to prove using the directly-expressible Liar sentence L in place of the Gödel sentence G . Gödel’s second incompleteness theorem, in contrast – that a consistent (classical) formal system cannot prove itself

consistent – fails to apply to GA, because the *habeas quid* or dynamic-typing rules above block the proof of Gödel’s key diagonalization lemma. This property of GA is in fact unsurprising in light of its close relationship to Kripke-inspired paracomplete logics, which were centrally motivated by circumventing Tarski’s closely-related undefinability theorem [Tarski, 1983] in order to allow a consistent logic to express its own truth predicate.

For rigorous metalogical analysis and consistency checking we have formalized BGA and GA with the Isabelle theorem prover [Nipkow et al., 2002], using its classical higher-order logic HOL as a mature meta-logic. A mechanically-verified consistency proof for the quantifier-free BGA fragment is complete, including for an optional primitive induction schema. This consistency proof uses largely-standard techniques to prove BGA’s inference rules truth-preserving, with respect to an operational semantics for BGA that is quite similar to (and in fact slightly simpler than) that of the classic minimalist programming language PCF [Plotkin, 1977]. Adding the quantifiers does not introduce inconsistency risks, *per se*, since the quantifiers are definable as computations within BGA, and need no further primitive rules or axioms. Full mechanically-checked proofs that GA’s quantification inference rules correctly *correspond* to these definable computations, however, are in progress but not yet complete. The main challenge is that these proofs require substantial tedious reflective reasoning, essentially the same issue that makes full mechanically-checked proofs of Gödel’s second incompleteness theorems nontrivial and rare [Paulson, 2014], although the key underlying techniques are now well-understood in principle.

This paper focuses solely on the fundamentals of consistent grounded or “dynamically-typed” formal reasoning in the presence of unconstrained recursion. We leave to future work many important further steps needed to make grounded deduction more richly expressive and comfortably usable for practical formal reasoning about diverse computations. Although the *habeas quid* principle and its semblance of dynamic typing is central to GA’s approach to recursion and termination proofs, GA’s “type system” as such has only two types, natural numbers and booleans. Richer grounded type systems would be useful, and appear straightforward for finitary types encodable as natural numbers such as tuples and lists. Infinitary types such as sets and real numbers present interesting and less-trivial challenges for future work. Similarly, implementing grounded deduction in an automated prover for the primary purpose of practical direct use, rather than primarily for metalogical analysis, is a task we have only started (again using Isabelle) and remains largely for future work.

In summary, this paper’s primary contributions are: (a) The first paracomplete formal deduction system for natural-number arithmetic that consistently permits direct expression of arbitrary Turing-complete computation via unconstrained, potentially non-terminating recursive definitions; (b) a computable approach to quantification in which universal and existential quantifiers are reflective computations expressible within BGA, a simpler quantifier-free formal system; (c) a mechanically-verifiable proof that BGA is consistent with respect to Isabelle’s higher-order logic HOL.

Reviewing note: There is a related publicly-available preprint that covers the topic of grounded deduction and grounded arithmetic at length, which is by a subset of the authors of this submission. In case reviewers might have encountered that preprint, we note that the preprint was intended as an exploratory idea paper, and has never been published or submitted for peer-reviewed publication.

The rest of this paper is organized as follows. Section 3 defines and informally analyzes the quantifier-free basic grounded arithmetic or BGA system, to which Section 4 then adds the computable quantifiers to produce the full first-order GA system. Section 5 summarizes the status of our mechanically-checked metalogical model and analysis of BGA and GA. Section 6 discusses potential future steps and implications of GA, Section 7 summarizes related work, and Section 8 concludes.

2 Background and Motivation

Reasoning formally about computation has always been challenging, due in part to a fundamental “impedance mismatch” between mathematical tradition and computation. In classical mathematical tradition, predicates are always either true or false, sets always decide their membership, and functions always decide their output for a given input. Computations in contrast may never terminate, and sometimes are not supposed to. Even after Church and Turing formulated clear mathematical models of computation, which later evolved into operational- and denotational-semantic models for reasoning about modern programming languages [Scott, 1982, Winskel, 1993], these useful models still require us to reason “at arm’s length” from the computations of interest: i.e., about program code *in a semantic model* built out of classical mathematical primitives, not about programs expressed directly in mathematical primitives. Domain-specific tools like Hoare logic [Hoare, 1969], process calculi [Milner, 1999], and separation logic [Reynolds, 2002] can help bridge this conceptual gap, but bring constraints and limitations of their own.

Even as proof assistants like Isabelle [Nipkow et al., 2002] and Coq/Rocq [Chlipala, 2013] have matured to the point of being usable for the mechanically-checked formal verification of significant software systems [Klein et al., 2014, Leroy et al., 2016], the challenges remain substantial. Few deployed systems, even mission-critical ones, are formally verified. The dependent type systems in recent proof assistants like Coq/Rocq [Chlipala, 2013], Lean [de Moura and Ullrich, 2021], and Agda [Bove et al., 2009] reduce the gap between reasoning and computation by allowing functional program code to be used both as a target for reasoning and within the reasoning process itself, accomplished in the type system via the Curry-Howard correspondence [Howard, 1980]. This code used for formal reasoning in these systems must still be provably terminating or strongly normalizing via the type system, however, which constrains expressiveness and limits the computational power of the expressible code.

A fundamental challenge facing formal reasoning is ensuring logical consistency. To be trustworthy, a useful formal system should not allow us to prove absolutely anything, which the classical principle of explosion allows us to do as soon as any proposition ‘ p ’ and its negation ‘ $\neg p$ ’ are both provable [Mendelson, 2015]. From Russell’s discovery that Frege and Cantor’s set theory was inconsistent [Whitehead and Russell, 2011], to the same discovery by Kleene and Rosser [Kleene and Rosser, 1935] about Curry’s and Church’s proposed systems for computational reasoning [Curry, 1930, Church, 1932], to Girard’s similar observation about early type theories in the Martin-Löf tradition [Girard, 1972], it remains extremely difficult to ensure that our formal systems are consistent. Most of the powerful dependent type theories lack any consistency proof, and even more-conservative type systems like that of Isabelle’s HOL, which in principle reduces to ZF set theory, still encounters subtle consistency issues due to convenient language extensions like overloading [Kunčar and Popescu, 2018].

The conventional wisdom, and the weight of experiential evidence so far, is that formal systems cannot permit direct expression of arbitrary computations – especially non-terminating ones – without falling into inconsistency. This is why Skolem’s primitive-recursive arithmetic (PRA) structurally constrains the forms of recursion it can express [Skolem, 1923, Mendelson, 2015], and similarly the typing and normalization constraints Church imposed on his simply-typed lambda calculus [Church, 1941] after his untyped formal system was found inconsistent [Kleene and Rosser, 1935]. But is ensuring termination and constraining computational expressiveness truly the *only* way we can admit computation directly into formal reasoning?

Logicians and philosophers have long explored theories of truth [Tarski, 1983, Kripke, 1975, Maudlin, 2006] as well as the liar [Beall, 2008, Simmons, 1993] and related paradoxes [Field, 2008]. There has been a particular effort in the philosophy of logic towards circumventing Tarski’s undefinability theorem [Tarski, 1983], which builds on Gödel’s incompleteness theorems [van Heijenoort, 2002, Mendelson, 2015] to show that a consistent, sufficiently-powerful *classical* formal system cannot precisely model its own semantics or prove itself consistent. This exploration has yielded two significant bodies of work in alternatives to classical logic [Field, 2008]: *paracomplete* systems [Kripke, 1975], which drop the classical expectation that every

$t = v$	variable reference	$\mathbf{T} \equiv (0 = 0)$	true constant
$ 0$	natural-number constant zero	$\mathbf{F} \equiv (0 = \mathbf{S}0)$	false constant
$ \mathbf{S} t$	natural-number successor	$a \mathbf{N} \equiv (a = a)$	number type
$ \mathbf{P} t$	natural-number predecessor	$p \mathbf{B} \equiv p \vee \neg p$	boolean type
$ \neg t$	logical negation	$p \wedge q \equiv \neg(\neg p \vee \neg q)$	conjunction
$ t \vee t$	logical disjunction	$p \rightarrow q \equiv \neg p \vee q$	implication
$ t = t$	equality of natural numbers	$p \leftrightarrow q \equiv (p \rightarrow q) \wedge (q \rightarrow p)$	biconditional
$ t ? t : t$	conditional evaluation	$a \neq b \equiv \neg(a = b)$	inequality
$ d(t, \dots, t)$	apply recursive definition		

(a) Abstract term syntax of BGA
(b) Metalogical shorthands for BGA

Table 1: Term syntax and metalogical shorthands for basic grounded arithmetic (BGA)

well-formed proposition must be either true or false, and *paraconsistent* systems [Priest, 2006], which relax the classical expectation that a formal system should be entirely consistent. Paraconsistent systems attempt instead to be more robust when inconsistency does occur, e.g., by limiting the explosion principle. We focus here on the paracompleteness alternative over paraconsistency, for the simple reason that we are not yet ready to give up on consistency as a seemingly-minimal baseline test for the trustworthiness of a formal system.

The paracomplete systems that have been proposed so far, however, even the experts who proposed them do not appear to be inclined to start using for everyday formal reasoning [Kripke, 1975, Field, 2008, Maudlin, 2006]. To our knowledge no paracomplete formal system has been developed sufficiently to be demonstrably usable for practical reasoning about computation in particular, even for computation only on natural numbers as in LCF [Scott, 1993, Milner, 1972] and PCF [Plotkin, 1977]. We thus raise the interdisciplinary question: can paracomplete logic be made usable for practical formal reasoning about computation? Despite its many limitations and as-yet-unknowns to be discussed in Section 6, we believe that GA provides an existence proof that the answer to this question is yes. We devote the rest of this paper to making this case.

3 Quantifier-free basic grounded arithmetic (BGA)

This section defines and analyzes Basic Grounded Arithmetic (BGA), a quantifier-free formal arithmetic that incorporates unrestricted, potentially-nonterminating recursive definitions and sufficient power to express Turing-complete computation. Consistently with prior quantifier-free systems like PRA [Mendelson, 2015, Skolem, 1923], the term “quantifier-free” means that BGA directly supports only the implicit top-level universal quantification of free variables, but not the explicit variable-binding quantifiers ‘ $\forall$ ’ or ‘ $\exists$ ’.

3.1 BGA term syntax

Table 2 summarizes the abstract term syntax of BGA. This term syntax is untyped, making no distinction between terms representing objects in the domain of discourse (natural numbers) and logical formulas. We could impose a static typing discipline, but none is formally necessary, and we avoid static typing to avoid potential confusion with the notion of dynamic typing more central to GA. As a hierarchical abstract syntax, this formulation ignores linearization considerations: we simply add parentheses and/or standard precedence rules to obtain an unambiguous linear-text syntax.

The formal syntax omits several operators that we will treat as non-primitive metalogical shorthands, summarized in Table 1b. The first two of these shorthands represent the boolean constants true (T) and false (F). The next two represent *dynamic type tests* for values of particular types, as we will examine shortly. The

Definition		Equality	
$\frac{s(\vec{x}) \equiv d(\vec{x})}{\frac{p\langle d(\vec{a}), \dots \rangle}{p\langle s(\vec{a}), \dots \rangle} \equiv IE}$	$\frac{a = b}{b = a} = S$	$\frac{a = b}{\frac{p\langle a, \dots \rangle}{p\langle b, \dots \rangle} = E}$	
Propositional logic			
$\frac{p}{\neg\neg p} \neg\neg IE$	$\frac{p \quad \neg p}{q} \neg E$	$\frac{p}{p \vee q} \vee I1$	$\frac{q}{p \vee q} \vee I2$
$\frac{\neg p \quad \neg q}{\neg(p \vee q)} \vee I3$	$\frac{p \vee q \quad p \vdash r \quad q \vdash r}{r} \vee E1$	$\frac{\neg(p \vee q)}{\neg p} \vee E2$	$\frac{\neg(p \vee q)}{\neg q} \vee E3$
Natural numbers			
$\frac{}{0 \text{ N}} 0I$	$\frac{a = b}{S(a) = S(b)} S=IE$	$\frac{a \neq b}{S(a) \neq S(b)} S \neq IE$	$\frac{a \text{ N}}{S(a) \neq 0} S \neq 0I$
$\frac{a \text{ N}}{P(S(a)) = a} P=I2$	$\frac{a \text{ N}}{P(a) \text{ N}} P^T IE$	$\frac{c \quad a \text{ N}}{(c ? a : b) = a} ?I1$	$\frac{\neg c \quad b \text{ N}}{(c ? a : b) = b} ?I2$
$\frac{p\langle 0, \dots \rangle \quad x \text{ N}, p\langle x, \dots \rangle \vdash p\langle S(x), \dots \rangle \quad a \text{ N}}{p\langle a, \dots \rangle} Ind$			
Structural rules			
$\frac{}{\Gamma, p \vdash p} H$	$\frac{\Gamma \vdash q}{\Gamma, p \vdash q} W$	$\frac{\Gamma, p, p \vdash q}{\Gamma, p \vdash q} C$	$\frac{\Gamma, p, q, \Delta \vdash r}{\Gamma, q, p, \Delta \vdash r} P$

Table 2: Inference rules for basic grounded arithmetic (BGA)

remaining shorthands define logical operators via the familiar equivalences valid for classical logic, which remain valid in GA – although they lead to different deduction rules.

3.2 Inference rules for deduction in BGA

Table 2 summarizes the logical inference rules comprising BGA, explained below in more detail.

3.2.1 Notation

Rules with a double line, such as $\neg\neg IE$, are *bidirectional rules*, representing an introduction rule when read normally with premise above the double line and conclusion below, and an elimination rule when flipped and read with premise below the double line and conclusion above. The definition substitution rule $\equiv IE$ is a *conditional bidirectional rule*: provided the premise above the single line to the left holds, the double-lined part to the right is applicable bidirectionally.

The notation ‘ $p\langle \cdot \rangle$ ’ represents a *syntactic template* that can express substitutions for free variables. In particular, if x denotes a variable, the notation ‘ $p\langle x \rangle$ ’ represents an otherwise-arbitrary term p having exactly one free variable x . If d is a term, the notation ‘ $p\langle d \rangle$ ’ represents same term p after replacing all instances of the free variable x with term d . The notation ‘ $p\langle x, \dots \rangle$ ’ indicates that the template term p may also contain other free variables in addition to x .

We also use the notation ‘ $\underline{n}$ ’ to denote the numeral in GA syntax constructed from some metalogical

natural number n . Specifically, we define numeral terms recursively as follows:

$$\underline{0} = 0 \quad \underline{n+1} = \mathbf{S}(n)$$

For brevity and intuitive clarity, only the structural rules explicitly mention lists or sets of background hypotheses (Γ, Δ). To expand the rest of the rules in the figure to account for background hypotheses, we simply prepend ‘ $\Gamma \vdash$ ’ to each premise or conclusion that does not already contain an entailment symbol ‘ $\vdash$ ’, and we prepend ‘ $\Gamma,$ ’ to hypothetical premises already containing an entailment symbol. In the induction rule *Ind*, the induction variable x must not be free in the background hypotheses Γ . If we consider hypotheses to be finite sets rather than lists, then the contraction (*C*) and permutation (*P*) rules become unnecessary and may be dropped.

We next examine GA’s inference rules in more detail, focusing primarily on important similarities with and differences from both classical and intuitionistic logic.

3.2.2 Logical negation

Modern formulations of classical logic typically use two inference rules to define logical negation: *negation introduction* and *double-negation elimination*. In brief, GA rejects classical negation introduction, but introduces the bidirectional rule $\neg\neg IE$ providing for both introduction and elimination of double negation. GA additionally includes a *negation-elimination* rule $\neg E$ directly expressing the classical law of non-contradiction, whose main pragmatic purpose is to close off lines of reasoning that are impossible by virtue of having contradictory premises.

Classical negation introduction embodies the principle of *reductio ad absurdum* or reduction to absurdity: if we hypothetically assume any proposition p , and from that assumption can derive a contradiction – i.e., both q and $\neg q$ for any proposition q – then we may conclude $\neg p$: i.e., that p must be false. This principle directly leads to the classical practices of proof and refutation by contradiction, and to the classical law of excluded middle (LEM): for any proposition p , either p or $\neg p$ must be true. Like intuitionism, GA rejects classical *reductio ad absurdum*, proof by contradiction, and LEM – not on the philosophical or constructivist grounds that motivated intuitionism [Heyting, 1975, Brouwer, 1981, Bishop, 1967], but instead for the pragmatic reason that strong *reductio ad absurdum* is inconsistent with the unconstrained recursive definitions we wish to permit in GA. The $\neg\neg IE$ rule distinguishes GA from intuitionism, however, which rejects the logical equivalence of a formula with its double negation.

The clearest illustration for why GA rejects LEM is the venerable *liar paradox* [Beall, 2008]: if I claim “I am lying”, am I lying or telling the truth? Unconstrained recursive definitions enable us to express the liar paradox simply and directly in GA, via the valid definition ‘ $L \equiv \neg L$ ’. If we were to admit such a definition in a classical system, we could hypothetically assume L , use L ’s definition to get $\neg L$, then use *reductio ad absurdum* with that contradiction to conclude $\neg L$ unconditionally (no longer hypothetically). In the same vein, we assume $\neg L$ to get L unconditionally, a logical inconsistency.

We can still use proof and refutation by contradiction in GA, but only via the following non-classical rules, which we can derive from the other primitive rules in Table 2 (see Section A.1):

$$\frac{p \text{ B} \quad p \vdash q \quad p \vdash \neg q}{\neg p} \quad \frac{p \text{ B} \quad \neg p \vdash q \quad \neg p \vdash \neg q}{p}$$

The first rule corresponds to classical negation introduction, except for the newly-added precondition ‘ $p \text{ B}$ ’: informally, that “ p is boolean.” With this precondition, GA demands *first* that p be proven to be “a thing” (the *habeas quid* principle) of the appropriate type (in this case boolean), *before* this particular term p may be used in refutation or proof by contradiction via these derived rules.

Notice from Table 1b that ‘ $p \text{ B}$ ’ in GA is simply a shorthand for ‘ $p \vee \neg p$ ’. Classically this is just the law of excluded middle (LEM), a tautology. These *habeas quid* preconditions could thus be added to classical

rules, but would always be trivially satisfied and hence pointless. GA, however, rejects the classical blanket assumption that any well-formed proposition p is either true or false, but instead demands a prior *proof* that p actually denotes a boolean value – *dynamically* by virtue of however the term p actually evaluates to a value, rather than statically based on p ’s syntactic structure alone. In GA, therefore, the shorthand ‘ $p \text{ B}$ ’ represents a *dynamic type check* that p denotes a terminating computation yielding a defined boolean value of either true or false.

Although the liar paradox is directly expressible in GA, these new *habeas quid* preconditions on contradiction proofs prevent the liar “paradox” from leading to inconsistency. To apply proof or refutation by contradiction on ‘ $L \equiv \neg L$ ’, in particular, we find we would *first* have to prove that L denotes a dynamically well-typed boolean value, i.e., that ‘ $L \text{ B}$ ’ or ‘ $L \vee \neg L$ ’ – but the only apparent way to do that would be to have already completed our proof by contradiction, via the rule we are still trying to justify invoking! In effect, the *habeas quid* preconditions on contradiction proofs in GA defuse the liar by erecting a *circular proof obligation* as a roadblock to proving that L denotes any value at all. L having no value is not a problem since GA is paracomplete and hence rejects at the outset the classical assumption that L must necessarily denote some boolean value.

3.2.3 Disjunction and conjunction

Modern formulations of classical logic typically include inference rules for the positive (non-negated) cases of both conjunction ($\wedge$) and disjunction ($\vee$). From these positive-case rules we can classically derive negative-case rules, defining when a conjunction or disjunction formula is *false*, from the positive-case rules via proof by contradiction.

As shown in Table 2, GA takes a different approach, treating disjunction as primitive and providing both positive-case *and* negative-case inference rules for it. We then define conjunction in terms of disjunction and negation using de Morgan’s laws (Table 1b). We can then derive the classical inference rules for conjunction based on this definition and the other primitive rules in Table 2, despite GA’s lack of strong proof by contradiction or LEM. This choice of primitive is largely arbitrary: we could as well consider conjunction to be primitive and derive disjunction from it.

Ultimately, GA’s rules for conjunction and disjunction are identical to those of classical logic, regardless of which we consider primitive. Conjunction and disjunction therefore remain duals respecting de Morgan’s laws, unlike intuitionistic systems, in which one of de Morgan’s laws fails.

3.2.4 Implication

GA defines *material implication* ($\rightarrow$) as a shorthand in Table 1b, in the same way material implication may be defined in classical logic. The lack of classical *reductio ad absurdum* in GA, however, affects the inference rules for implication that we can derive in GA (Section A.3):

$$\frac{p \text{ B} \quad p \vdash q}{p \rightarrow q} \rightarrow I \qquad \frac{p \rightarrow q \quad p}{q} \rightarrow E$$

GA’s implication elimination rule $\rightarrow E$ is identical to the corresponding classical and intuitionistic rule, also known as *modus ponens*. GA’s implication introduction rule $\rightarrow I$, however, adds the *habeas quid* or dynamic typing precondition ‘ $p \text{ B}$ ’, just as in the earlier contradiction proof rules.

Consider Curry’s paradox, which we might express informally via the claim “if I am telling the truth then pigs fly” – or formally via a recursive definition of the form ‘ $C \equiv C \rightarrow P$ ’ for any arbitrary proposition P . Given this definition and the classical or intuitionistic inference rules for material implication, we can hypothetically assume C , unwrap its definition to $C \rightarrow P$, and use *modus ponens* ($\rightarrow E$) to get P hypothetically. But since we have built a hypothetical chain from C to P , classical or intuitionistic implication introduction

$\rightarrow I$ allow us to infer ‘ $C \rightarrow P$ ’ now unconditionally. Substituting C ’s definition again, we get the unconditional truth of C , and then by *modus ponens* again, the unconditional truth of the arbitrary proposition P (e.g., “pigs fly”).

Curry’s paradox illustrates why intuitionistic systems cannot tolerate unrestricted recursive definitions, despite having rejected strong contradiction proofs and the classical LEM. If we attempt to carry out this proof in GA, however, we find that the new *habeas quid* precondition ‘ $p \text{ B}$ ’ in GA’s implication introduction rule $\rightarrow I$ again presents a circular proof obligation. In order to invoke this rule in our attempt to prove C true, we must *first* have proven at least that C has some boolean value – but we find no way to do this without having already applied the implication introduction rule we are still trying to justify invoking. Like the liar paradox, therefore, GA simply leaves Curry’s paradox as another statement that harmlessly denotes no provable value, thereby avoiding inconsistency.

We define the material biconditional ‘ $p \leftrightarrow q$ ’ as ‘ $(p \rightarrow q) \wedge (q \rightarrow p)$ ’, exactly as in classical logic (Table 1b). The inference rules we derive from this definition in GA, however, differ from classical logic by imposing *habeas quid* preconditions on both p and q in the introduction rule:

$$\frac{p \text{ B} \quad q \text{ B} \quad p \vdash q \quad q \vdash p}{p \leftrightarrow q} \leftrightarrow I \quad \frac{p \leftrightarrow q \quad p}{q} \leftrightarrow E1 \quad \frac{p \leftrightarrow q \quad q}{p} \leftrightarrow E2$$

3.2.5 Equality

Equality in GA has the same rules for symmetry, transitivity, and substitution of equals for equals as in classical logic with equality. Transitivity is not shown in Table 2 because it is derivable from the substitution rule $=E$. Conspicuously missing from GA, however, is reflexivity.

As with the dynamic booleanness test ‘ $p \text{ B}$ ’, GA applies the *habeas quid* principle to natural numbers with the dynamic type-check ‘ $a \text{ N}$ ’. Only natural numbers yield a defined result in GA when compared for equality, so testing whether a is a natural number amounts to testing whether a is equal to itself, as shown in Table 1b. We cannot just assume some GA term a yields a natural number by static typing, but must *prove* dynamically that it does so, by satisfying ‘ $a \text{ N}$ ’ or ‘ $a = a$ ’.

3.2.6 Natural numbers

The rules for natural numbers in GA broadly follow the classical Peano axioms, with small but important differences. The $0I$ rule asserts that 0 is a natural number. The bidirectional equality rule $S=IE$ states that a and b are equal precisely when their respective successors are. Setting b to a in this rule gives us the fact that the successor of any natural number a is a natural number, and the bidirectional rule’s reverse direction asserts that successor is injective. The $S \neq IE$ rule asserts the same properties for not-equals, which in GA are not derivable from the equality rules due to the lack of classical contradiction proofs. The $S \neq 0I$ rule states that successor always generates new numbers that never “wrap” to zero as they would in modular arithmetic.

BGA also includes a primitive predecessor operation P , and a primitive conditional-evaluation or “if/then/else” operator, for which we use the concise C-like syntax ‘ $c ? a : b$ ’. These operators need not be primitive in classical Peano arithmetic or PRA, and similarly need not be primitive in full GA with quantifiers (Section 4), but quantifier-free BGA appears to need them, or some equivalent, to “bootstrap” expression of full Turing-complete computation. GA’s set of natural-number computation primitives perhaps unsurprisingly lines up closely with the arithmetic-oriented formal system LCF [Scott, 1993] and the minimal programming language PCF [Plotkin, 1977], although GA’s non-classical logic takes a formal-reasoning direction quite different from either LCF or PCF.

GA’s mathematical induction rule *Ind* is conventional except for new *habeas quid* preconditions. To prove inductively that a predicate p holds for whatever value an arbitrary term a denotes, *Ind* first requires not only base-case and inductive-step proofs, but also a proof that term a actually terminates and yields a

natural number. The required inductive-step proof is now allowed two hypotheses, however: not only that p holds for the induction variable x but also that x is a natural number.

3.3 Expressing recursive computation in BGA

Having summarized the syntax and deduction rules comprising BGA, we now briefly explore their usefulness for reasoning about arithmetic computations on natural numbers.

3.3.1 Proving termination by dynamic typing

Static type systems for informal programming languages traditionally prove only a safety property – namely that a typed term never produces any value *other than* one of the correct type. These type systems usually prove nothing about liveness or termination, i.e., whether the typed term ever actually produces a value. Type systems for formal languages traditionally preserve this static paradigm, but since classical formal systems readily fall into inconsistency if defined functions fail to terminate, recursive definitions must typically be justified by a termination proof that relies on a line of reasoning orthogonal to the type system: e.g., by building a well-founded set, ordinal, or type “big enough” to contain the desired function and selecting the function from it. GA’s dynamic typing proofs, in contrast, double as termination proofs.

As a simple but illustrative example, let us consider the process of constructing natural numbers. The premise-free rule $0I$ allows us to conclude ‘ $0 \mathbf{N}$ ’: i.e., that 0 is a natural number. What this one-line proof’s conclusion really states, in fact, is that the term ‘0’ is a *terminating computation* yielding a natural number. Using the $S=IE$ rule with a and b set to zero, along with the shorthand ‘ $a \mathbf{N} \equiv a = a$ ’ from Table 1b, we can prove that the term ‘ $S(0)$ ’ – i.e., 1 – is likewise a terminating computation yielding a natural number. For any particular natural number n , we can apply the $S=IE$ rule n times in this way to prove that there is a terminating computation yielding the natural number n .

Longer proofs in this series, for larger natural numbers, build on shorter termination proofs for smaller natural numbers. These proofs in effect just symbolically perform the construction, dynamic typing, and termination-proving of a particular natural number in reverse, starting with smaller natural numbers, and proceeding to larger numbers built on prior smaller ones. We can thus see how proving termination and dynamic typing in GA may be equivalently viewed as reverse symbolic execution of computations, as will become clearer in Section 3.4 on BGA’s operational semantics.

3.3.2 Generalized dynamic typing by induction

Each of the above trivial number-construction examples constitute termination proofs only for *specific* natural numbers, of course, and are not general proofs applying to all natural numbers. We can use recursive definitions and the induction rule Ind to construct more general dynamic typing and termination proofs, however (Section A.5).

Consider for example the following recursive definition:

$$\text{even}(n) \equiv n = 0 ? 1 : 1 - \text{even}(\mathbf{P}(n))$$

This recursive definition need not be justified before being introduced and used in GA, but we cannot assume it is dynamically well-typed or terminating until we have proven it as such. Such a dynamic proof is easy using induction, however. Assuming we have already proven the constant ‘ $1 \equiv \mathbf{S}(0)$ ’ and subtraction terminating, we need to prove ‘ $n \mathbf{N} \vdash \text{even}(n) \mathbf{N}$ ’ by induction on n . In the base case $n = 0$, the conditional-evaluation ‘?’ evaluates to 1, reducing to our existing proof of ‘ $1 \mathbf{N}$ ’. In the inductive step, we can assume ‘ $\text{even}(n) \mathbf{N}$ ’ and must prove ‘ $\text{even}(\mathbf{S}(n)) \mathbf{N}$ ’. We reach this conclusion by applying the definition and the

Propositional logic typing rules			
$\frac{p \text{ B}}{\neg p \text{ B}} \neg TIE$	$\frac{p \text{ B} \quad q \text{ B}}{p \vee q \text{ B}} \vee TI$	$\frac{p \vee q \text{ B}}{(p \text{ B}) \vee (q \text{ B})} \vee TE$	
$\frac{p \text{ B} \quad q \text{ B}}{p \wedge q \text{ B}} \wedge TI$	$\frac{p \wedge q \text{ B}}{(p \text{ B}) \wedge (q \text{ B})} \wedge TE$	$\frac{p \text{ B} \quad q \text{ B}}{p \rightarrow q \text{ B}} \rightarrow TI$	$\frac{p \rightarrow q \text{ B}}{(p \text{ B}) \rightarrow (q \text{ B})} \rightarrow TE$
$\frac{p \text{ B} \quad q \text{ B}}{p \leftrightarrow q \text{ B}} \leftrightarrow TI$	$\frac{p \leftrightarrow q \text{ B}}{p \text{ B}} \leftrightarrow TE1$	$\frac{p \leftrightarrow q \text{ B}}{q \text{ B}} \leftrightarrow TE2$	
Natural number typing rules			
$\frac{a \text{ N}}{\text{S}(a) \text{ N}} \text{S} TIE$	$\frac{a \text{ N}}{\text{P}(a) \text{ N}} \text{P} TIE$	$\frac{a \text{ N} \quad b \text{ N}}{a = b \text{ B}} = TI$	$\frac{c \text{ B} \quad a \text{ N} \quad b \text{ N}}{c ? a : b \text{ N}} ? TI$

Table 3: Typing rules for BGA derivable from the primitive rules in Table 2

primitive rules $S \neq 0I$ and $P = I$ in Table 2, and the prior termination proofs for 1 and subtraction, completing the inductive proof. The full termination theorem is stated and proven in Theorem A.14 in the appendix.

3.3.3 Typing rules

Statically-typed formal systems usually need many primitive typing rules. Most of GA’s typing rules for the basic logical and natural-number operations, in contrast, are derivable from the primitive computational inference rules. Table 3 summarizes a non-exhaustive set of such derivable typing rules for BGA, leaving details of these derivations to Section A.6.

3.3.4 Primitive recursion

The *primitive-recursive functions* are central to Skolem’s formal system now known as PRA [Skolem, 1923, Mendelson, 2015]. These primitive-recursive functions take natural-number arguments and yield natural numbers, always terminate, and are constructed inductively from four standard *initial functions* via two composition operations: function substitution and primitive recursion.

Pragmatically, primitive recursion is powerful enough to express essentially all practical computations up through exponential time complexity, but falls short of general-recursive or Turing-complete computation by failing to express super-exponential functions such as Ackermann’s function [Ackermann, 1928, Kleene, 1952].

We can directly express any primitive-recursive function in GA, unsurprisingly since primitive recursion is just a structurally-constrained form of general recursion. We can moreover prove in GA that any such primitive-recursive function always terminates, by following the primitive-recursive function composition structure, proving the termination of simpler primitive-recursive functions first followed by the more-complex functions composed from them, and using an inductive proof like that in the *even* example above for functions composed via primitive recursion. Although we mention this property only informally here without rigorous proof, it should be clear that GA is at least as powerful as PRA, in terms of both expressiveness and termination-proving capability.

3.3.5 General recursion

Any general-recursive, Turing-complete function may be expressed via a combination of primitive-recursive functions and a single instance of an existential quantifier or any equivalent “unbounded search” capability [Kleene, 1952, Mendelson, 2015]. Intuitively, primitive-recursive functions are powerful enough to advance a Turing machine or equivalent computational model by one step, or by any concretely-specified number of steps. The remaining crucial ingredient of unbounded search advances the machine to the first step at which it terminates, if it ever terminates. Both primitive recursion and unbounded search are readily expressible via recursive definitions in GA.

Concretely, assume $T(x, y)$ is a primitive-recursive function that takes an input x and a step count y , returns $1 + z$ if the computation terminates within y steps with output z , and returns 0 if the computation has not yet terminated within y steps. The recursive function $C(x, y) \equiv T(x, y) \neq 0 \ ? \ \mathbf{P}(T(x, y)) : C(x, \mathbf{S}(y))$, definable in GA, performs an unbounded search for a y for which the underlying step function T terminates. Thus, $C(x, 0)$ denotes the computation’s result provided it terminates; otherwise $C(x, 0)$ represents a non-terminating computation and denotes no value.

For any particular input x on which the computation represented by C terminates with some result z , it is provable in GA that $C(x, 0)$ indeed terminates with z , essentially by executing the computation itself in reverse as in the simple examples above. If the computation does not terminate on input x , however, we will be unable to prove anything about $C(x, 0)$ – at least not directly.

GA is thus capable of *expressing* any Turing-complete computation via recursive definitions, and when a recursive computation terminates on a particular input, GA can prove that it does so. We leave open for now, however, questions such as: (a) can GA ever prove that computations *do not* terminate? (b) how general is GA’s termination-proving capability, for more abstractly-specified inputs? We will return to these questions in Section 4 when we explore reflection and quantifiers in GA.

3.4 Operational semantics of BGA terms

Not only is any recursive computation directly expressible in GA, but conversely every well-formed GA term expresses a computation – though of course not necessarily a terminating one. We can formalize this fact by assigning GA terms a computational semantics. We have explored approaches based on both operational and denotational semantics, but we focus here on operational semantics because the reasoning is simpler and adequate for our present purposes.

We could in fact easily convert any GA term into a program in the minimalistic language PCF [Plotkin, 1977], and rely on an existing operational or denotational semantics for PCF. BGA is simpler than PCF, however, in that BGA has no higher-order functions that can be passed to and returned from other functions, but rather assumes only an arbitrary-but-fixed set of recursive function definitions. We could of course extend BGA to include PCF’s expressive power and beyond, but need not in order to achieve our main purposes here, and hence choose to keep BGA as simple as reasonably feasible.

BGA’s operational semantics has two parameters affecting its operation: an assignment A mapping variables to concrete values, and a definition list D of recursive definitions available.

3.4.1 Assignments

We consider a *variable symbol* v in BGA to have the form $\mathbf{v}_i$, indexed by any natural number i . An assignment A is a function mapping each variable $\mathbf{v}_i$ either to a natural number n , or to a distinguished *bottom* symbol ‘ $\perp$ ’ denoting no assigned value.

$\frac{a_{j< \vec{a} } \Downarrow n_j \quad D_i(\vec{n}) \Downarrow m}{\mathbf{d}_i(\vec{a}) \Downarrow m}$	$\frac{a \Downarrow n \quad b \Downarrow n}{a = b \Downarrow 1}$	$\frac{a \Downarrow n \quad b \Downarrow m \quad n \neq m}{a = b \Downarrow 0}$	$\frac{A(\mathbf{v}_j) = n}{\mathbf{v}_j \Downarrow n}$
$\frac{}{0 \Downarrow 0}$	$\frac{a \Downarrow n}{\mathbf{S}(a) \Downarrow n+1}$	$\frac{a \Downarrow 0}{\mathbf{P}(a) \Downarrow 0}$	$\frac{a \Downarrow n+1}{\mathbf{P}(a) \Downarrow n}$
	$\frac{c \Downarrow 1 \quad a \Downarrow n}{c ? a : b \Downarrow n}$	$\frac{c \Downarrow 0 \quad b \Downarrow n}{c ? a : b \Downarrow n}$	
$\frac{p \Downarrow 1}{\neg p \Downarrow 0}$	$\frac{p \Downarrow 0}{\neg p \Downarrow 1}$	$\frac{p \Downarrow 1}{p \vee q \Downarrow 1}$	$\frac{q \Downarrow 1}{p \vee q \Downarrow 1}$
	$\frac{p \Downarrow 0 \quad q \Downarrow 0}{p \vee q \Downarrow 0}$		

Table 4: Big-step structural operational semantics (BSOS) for reduction in GA

3.4.2 Recursive definitions

A *definition symbol* d has the form $\mathbf{d}_i$, indexed by a natural number i . A *definition body* is simply an arbitrary BGA term in the syntax defined in Section 3.1. A *definition list* is a finite list of definition bodies. For all integers $0 \leq i < |D|$, definition body D_i represents the body term associated with definition symbol $\mathbf{d}_i$. The *arity* k_i of definition D_i is the least natural number greater than the natural-number index j of any free variable $\mathbf{v}_j$ appearing in body D_i .

Each definition D_i in effect represents a computable function taking k_i natural-number arguments and returning a natural number. Each position i in the definition list D thus represents the definition ‘ $\mathbf{d}_i(\mathbf{v}_0, \dots, \mathbf{v}_{k_i-1}) \equiv D_i(\mathbf{v}_0, \dots, \mathbf{v}_{k_i-1})$ ’. We place no restrictions on the definition symbols that may appear within definition bodies – in particular, no requirement that definitions be defined before being referenced – so definitions may be singly- and mutually-recursive. GA terms may even contain undefined definition symbols $\mathbf{d}_i$ for $i \geq |D|$. Our semantics will provide no way to reduce such a symbol, so an undefined definition symbol will simply denote a nonterminating computation ($\perp$).

3.4.3 Reduction of BGA terms

Table 4 concisely presents a big-step structural operational semantics, or BSOS, for BGA terms [Winskel, 1993]. These reduction rules specify inductively how more complex GA terms may reduce to simpler ones, with some GA terms eventually reducing to a concrete natural number. For simplicity we encode the boolean constants T and F as the natural numbers 1 and 0, respectively. There is nothing new or special about BGA’s operational semantics; we present it only as a tool for metalogical reasoning about BGA.

BGA’s operational semantics uses the assignment parameter A only to reduce variable references, and only for variables that A maps to a natural number. Variables that A maps to $\perp$ can never reduce, which may prevent the computation from terminating (reducing to any value) under this assignment.

The reduction rule for recursive definitions uses call-by-value semantics, reducing an invocation of definition $\mathbf{d}_i$ with actual parameters $\vec{a} = a_1, \dots, a_{k_i}$ to result w only once each actual-parameter term a_j has reduced to a concrete value v_j , and upon substituting these values for the free variables in definition body D_i , the result reduces to w . Other call semantics would likely work, but we found that call-by-value semantics simplifies the truth-preservation reasoning in the next section.

An important property that BGA’s BSOS has, like those of many sequential languages, is that the semantic reduction relation is *single-valued* or *deterministic*: if ‘ $t \Downarrow v_1$ ’ and ‘ $t \Downarrow v_2$ ’, then ‘ $v_1 = v_2$ ’. This property is easily provable by rule induction (see Section A.7 for details).

3.5 Proving BGA’s consistency by truth preservation

A standard technique in metamathematics [Kleene, 1952] and particularly model theory [Mendelson, 2015, Butten and Walsh, 2018], in a semantic modeling tradition launched by Tarski [Tarski, 1983], is to prove a set of logical inference rules *truth-preserving* with respect to a particular semantic model of the formal system in question. If a proof in the target logic consists solely of truth-preserving steps or *judgments*, then we can deduce that the proof can only lead to a conclusion that is true under the semantic model – assuming, of course, that the metalogic in which we define and analyze this model is itself both powerful enough and trustworthy (e.g., hopefully at least consistent). By proving that the inference rules lead only to proofs of true statements, we can infer that the inference rules *cannot* lead to proofs of false statements, thereby proving the target logic’s consistency – at least relative to the consistency of our metalogic.

Standard metalogical practice is to prove a logic’s truth preservation with respect to a model constructed via classical mathematical tools such as sets and relations. We will diverge from this standard practice, however, by using a *computational* model – namely BGA’s operational semantics – to prove that BGA’s inference rules preserve truth in this computational model. Since BGA’s operational semantics above encodes the true constant $\top$ as the natural number 1, we are effectively proving that BGA’s inference rules safely navigate between terms that, under suitable conditions to be defined precisely below, denote computations that always terminate and reduce to the value 1.

3.5.1 Satisfaction of terms and judgments

In Hilbert-style natural-deduction systems of the type we defined for BGA in Section 3.2 and Table 2, an *inference rule* consists of zero or more *premises* and a single *conclusion*. Premises and conclusions are *judgments* or *entailments* of the form ‘ $\Gamma \vdash c$ ’ (or “ Γ entails c ”), where Γ is a list of *antecedents* or *hypotheses* and c is the *consequent*. Intuitively, a judgment expresses a metalogical claim that under whatever conditions make every hypothesis in Γ true, the consequent c is likewise true under the same conditions.

An assignment A *satisfies* BGA term t under definition list D if t reduces to 1 ($t \Downarrow 1$) under A and D via the operational semantics above. A judgment ‘ $\Gamma \vdash c$ ’ is *true* under definitions D if every assignment A that satisfies all hypotheses in Γ under D also satisfies conclusion c under D . An inference rule is *truth preserving* for BGA if, for any definitions D and any instance of the rule whose premises are true judgments, the instantiated rule’s conclusion is likewise a true judgment.

Entailment and truth preservation are non-computable notions with respect to the (computable) BGA target logic, but this is not a problem since the entailment or turnstile ‘ $\vdash$ ’ is a metalogical, not target-logic, construct. Since material implication ($\rightarrow$) is a target-logic operator and is computable in BGA, the distinction between material implication and metalogical entailment is pragmatically more crucial in GA than in classical or intuitionistic logic. Whereas the classical inference rules for implication allow free interchange between the judgments ‘ $a \vdash b$ ’ and ‘ $\vdash a \rightarrow b$ ’, for example, this is not true of BGA due to the *habeas quid* precondition for implication introduction (Section 3.2.4).

3.5.2 Truth preservation examples

We leave details of the truth-preservation proofs for BGA to Section A.8, and only briefly sketch a few illustrative examples here.

To prove the bidirectional inference rule $\neg\neg IE$ in Table 2 truth preserving, for example, we must show that under any assignment A satisfying all hypotheses Γ under definitions D , if term p evaluates to 1 then so does ‘ $\neg\neg p$ ’, and vice versa. In the forward direction, we simply apply two additional reduction steps to bring the assumed evaluation ‘ $p \Downarrow 1$ ’ to ‘ $\neg\neg p \Downarrow 1$ ’. In the reverse direction, we need lemmas proving that the *only* way we get to ‘ $\neg p \Downarrow 1$ ’ is from ‘ $p \Downarrow 0$ ’, and similarly for ‘ $\neg p \Downarrow 0$ ’ from ‘ $p \Downarrow 1$ ’. We prove both of these lemmas by case analysis on the available reductions.

To prove BGA’s negation-elimination or non-contradiction rule $\neg E$ truth preserving, we must show that any assignment A that satisfies all hypotheses in Γ and makes both ‘ p ’ and ‘ $\neg p$ ’ reduce to 1, A also satisfies the arbitrary conclusion q . But for ‘ $\neg p$ ’ to reduce to 1, p would have to reduce to 0 as well as to 1. Because BGA’s operational semantics is single-valued, this is impossible. We thus trivially satisfy the rule’s conclusion, by applying the law of non-contradiction metalogically.

As a negative example of a rule we *cannot* prove truth preserving with respect to BGA’s operational semantics, because it isn’t, consider the classical implication-introduction rule $\rightarrow I$, without BGA’s *habeas quid* precondition ‘ $p \mathbf{B}$ ’ (Section 3.2.4). We wish to prove that any assignment A satisfying all hypotheses in Γ reduces the BGA term ‘ $p \rightarrow q$ ’ to 1, but this is just shorthand for ‘ $\neg p \vee q$ ’ (Table 1b). If p reduces to 0 or q reduces to 1 then the conclusion is trivially satisfied. If p reduces to 1 then we can use the rule’s classical premise ‘ $\Gamma, p \vdash q$ ’ to infer that q likewise reduces to 1. But p is an arbitrary computation, which might reduce to a non-boolean natural number, or might never reduce at all and thus denote $\perp$. We are thus stuck and unable to prove truth preservation for this case.

To prove BGA’s mathematical induction rule *Ind* truth preserving, we must show that for any assignment A satisfying the hypotheses in Γ , in which the induction variable x is not free, the rule’s conclusion is true whenever its three premises are true. By the rule’s third *habeas quid* premise ‘ $\Gamma \vdash a \mathbf{N}$ ’, we infer that the arbitrary BGA term a reduces to some arbitrary but fixed natural number n . We prove metalogically by induction on n that A satisfies the rule’s conclusion ‘ $\Gamma \vdash p(a, \dots)$ ’. The base case of $n = 0$ is directly satisfied by the rule’s first premise ‘ $\Gamma \vdash p(0, \dots)$ ’. For the metalogical inductive step, we can assume the rule’s conclusion is satisfied when we substitute the numeral for n for the induction variable x , and must prove that it is also satisfied when we substitute the numeral for $n + 1$ for x . The inductive step’s starting assumptions enable us to prove that A satisfies the second premise’s additional hypotheses ‘ $x \mathbf{N}$ ’ and ‘ $p(x, \dots)$ ’ under these substitutions, so the second premise is applicable, allowing us to infer that A likewise satisfies ‘ $p(S(x), \dots)$ ’, thereby completing the induction step. The induction inference rule is therefore truth preserving.

4 Computable quantification in full grounded arithmetic (GA)

Although we can perform substantial useful reasoning in a system like PRA or BGA with only implicit top-level quantification of free variables, the addition of explicit quantifiers adds a great deal of desirable expressive power. Their presence also typically makes metalogical reasoning about a formal system much more complex, including for purposes of verifying the system’s consistency.

4.1 Inference rules for quantification

Table 5 shows the inference rules for both the positive and negative cases of universal and existential quantification. Adding these rules to those of BGA yields the full GA system.

In contrast with intuitionistic logic, the classical equivalences $\forall x p(x, \dots) \equiv \neg \exists x \neg p(x, \dots)$ and $\exists x p(x, \dots) \equiv \neg \forall x \neg p(x, \dots)$ remain valid in GA, despite the rules’ differences described in more detail below. As a result, using these equivalences, the existential quantifier may be defined in terms of the universal quantifier, or vice versa. Alternatively, the negative-case rules for each quantifier may be derived from the opposite quantifier’s positive-case rules.

As usual, these rules correspond to their classical equivalents, except for the addition of *habeas quid* or dynamic type tests. To introduce a universal quantifier ‘ $\forall x p(x, \dots)$ ’ via rule $\forall I1$, for example, we must first have a proof of the predicate p in which some variable x is free. This proof can assume nothing about x except that it is a natural number, via the ‘ $x \mathbf{N}$ ’ hypothesis. As in classical logic, the rule $\forall E1$ eliminates or instantiates a universal quantifier by showing that predicate p holds when arbitrary term a is substituted for

$\frac{x \text{ N} \vdash p\langle x, \dots \rangle}{\forall x p\langle x, \dots \rangle} \forall I1$	$\frac{\forall x p\langle x, \dots \rangle \quad a \text{ N}}{p\langle a, \dots \rangle} \forall E1$
$\frac{a \text{ N} \quad \neg p\langle a, \dots \rangle}{\neg \forall x p\langle x, \dots \rangle} \forall I2$	$\frac{\neg \forall x p\langle x, \dots \rangle \quad x \text{ N}, \neg p\langle x, \dots \rangle \vdash q\langle \dots \rangle}{q\langle \dots \rangle} \forall E2$
$\frac{a \text{ N} \quad p\langle a, \dots \rangle}{\exists x p\langle x, \dots \rangle} \exists I1$	$\frac{\exists x p\langle x, \dots \rangle \quad x \text{ N}, p\langle x, \dots \rangle \vdash q\langle \dots \rangle}{q\langle \dots \rangle} \exists E1$
$\frac{x \text{ N} \vdash \neg p\langle x, \dots \rangle}{\neg \exists x p\langle x, \dots \rangle} \exists I2$	$\frac{\neg \exists x p\langle x, \dots \rangle \quad a \text{ N}}{\neg p\langle a, \dots \rangle} \exists E2$

Table 5: Quantification inference rules for full grounded arithmetic (GA)

the quantified variable x – but unlike in classical logic, only provided a has been shown to denote a natural number, as the rule’s second premise ‘ $a \text{ N}$ ’ demands.

To introduce an existential quantifier ‘ $\exists x p\langle x, \dots \rangle$ ’ via $\exists I1$, we must first show not only that predicate p holds when the quantified variable x is replaced with arbitrary term a (in which x cannot occur free), as usual in classical logic, but also that term a in fact denotes a natural number (‘ $a \text{ N}$ ’). The rule $\exists E1$ eliminates an existential quantifier in favor of a chain of reasoning leading to the arbitrary conclusion q from two hypotheses: that free variable x represents some arbitrary but fixed natural number (‘ $a \text{ N}$ ’), and that predicate p holds on that natural number.

As usual, the quantification variable x cannot occur free in the background hypotheses Γ , which Table 5 leaves implicit for conciseness.

4.2 Quantifiers as general-recursive computations

In Peano arithmetic based on classical first-order logic, the arithmetic hierarchy [Kleene, 1943, Mostowski, 1947, Moschovakis, 2016] formally classifies statements only expressible with some minimum number of alternating universal and existential quantifiers. In classical first-order logic, therefore, the quantifiers appear to add fundamental and non-reducible reasoning power, at least assuming that Peano arithmetic is consistent.

The situation is strikingly different in grounded reasoning, however. With the addition of the *habeas quid* conditions in the rules above, GA’s quantifiers become ordinary computations expressible in any Turing-complete language, including in the term language of BGA. We therefore need not consider either quantifier to be primitive in GA. We instead treat both quantifiers as metalogical shorthands for nontrivial but otherwise-standard computations already expressible via recursive definitions in BGA.

4.2.1 Gödel coding and reflection

To define and reason about computable quantifiers in GA, we will use the same tools of reflection that Gödel introduced in his incompleteness theorems [van Heijenoort, 2002, Mendelson, 2015].

Omitting the details of these now-standard practices, we define natural-number encodings or *Gödel codes* for BGA’s primitive term syntax (Table 1a), for BGA judgments of the form ‘ $\Gamma \vdash c$ ’, and for BGA proofs consisting of a finite list of judgments. We use the common *Quine quote* notation ‘ $\ulcorner a \urcorner$ ’ to represent the Gödel code of BGA term or proof a [Quine, 1982]. We define primitive-recursive functions to test whether a natural number n encodes a syntactically well-formed BGA term or proof, respectively. Finally, we define a primitive-recursive proof-checking function $C(n, m)$, which returns 1 if n encodes a valid BGA proof that

correctly follows the BGA deduction rules presented earlier in Table 2 and concludes with a valid judgment encoded by m . Otherwise, $C(n, m)$ returns 0.

4.2.2 Strictly-positive quantifiers

As an intermediate step towards the full computable quantifiers, we first define computations representing *strictly-positive* existential and universal quantifiers, via the following two primitive-recursive functions E^+ and A^+ :

$$\begin{aligned} E^+(v, p, 0) &\equiv 0 \\ E^+(v, p, \mathbf{S}(s)) &\equiv E^+(v, p, s) \neq 0 \vee C(L(s), \ulcorner p(R(s), \dots) \urcorner) \neq 0 ? 1 : 0 \\ A^+(v, p, 0) &\equiv 0 \\ A^+(v, p, \mathbf{S}(s)) &\equiv A^+(v, p, s) \neq 0 \vee C(s, \ulcorner x \mathbf{N} \vdash p(x, \dots) \urcorner) \neq 0 ? 1 : 0 \end{aligned}$$

The $L(s)$ and $R(s)$ used in this definition are primitive-recursive functions that decompose their natural-number argument s into a Cantor pairing $\langle l, r \rangle$ and extract components l and r , respectively.

$E^+(v, p, s)$ takes Gödel-coded variable v , predicate p , and step count s . E^+ effectively performs a bounded search for some step count $s' = \langle l, r \rangle < s$ representing an encoded BGA proof l of the judgment $\ulcorner p(\underline{r}, \dots) \urcorner$.

If there is a concrete natural number n that causes $p(\underline{n}, \dots)$ to evaluate to true, and there is a BGA proof P of this fact, then $E^+(\ulcorner x \urcorner, \ulcorner p(x, \dots) \urcorner, s) = 1$ for all $s > \langle \ulcorner P \urcorner, n \rangle$. Since any concrete terminating recursive computation may simply be evaluated in reverse to construct a proof of its termination in BGA (Section 3.3.5), only the existence of the satisfying natural number n is really in question here. If no such satisfying n exists, then $E^+(\ulcorner x \urcorner, \ulcorner p(x, \dots) \urcorner, s) = 0$ for all s .

Although E^+ itself is a primitive-recursive function, we can view it as a *step function* defining a general-recursive computation (see Section 3.3.5) that eventually, for some step count s , terminates if any n satisfying the BGA predicate p exists, and otherwise simply never terminates.

The primitive-recursive $A^+(v, p, s)$ function analogously returns 1 if any $s' < s$ encodes a BGA proof of the judgment $\ulcorner x \mathbf{N} \vdash p(x, \dots) \urcorner$, and A^+ returns 0 otherwise. We leave x as an arbitrary free variable as part of p 's Gödel code in this case, but the explicit hypothesis $\ulcorner x \mathbf{N} \urcorner$ allows the sought-after BGA proof to assume that the free variable x denotes a natural number. As with E^+ , we can view A^+ as a step function indirectly defining a general-recursive computation that eventually, for some step count s , terminates if there is a quantifier-free BGA proof of $p(x, \dots)$ when we leave x denoting an arbitrary-but-fixed natural number; otherwise the computation never terminates.

4.2.3 Two-sided quantifiers

Building on the above primitive-recursive functions, we now define general-recursive functions expressing the full “two-sided” semantics of GA’s quantifiers, which can terminate yielding both true and false results. We define the following recursive functions in BGA:

$$\begin{aligned} E(v, p, s) &\equiv E^+(v, p, s) \neq 0 ? \mathbf{T} : A^+(v, \ulcorner \neg p \urcorner, s) \neq 0 ? \mathbf{F} : E(v, p, \mathbf{S}(s)) \\ A(v, p, s) &\equiv A^+(v, p, s) \neq 0 ? \mathbf{T} : E^+(v, \ulcorner \neg p \urcorner, s) \neq 0 ? \mathbf{F} : A(v, p, \mathbf{S}(s)) \end{aligned}$$

These definitions gloss over minor details with an abuse of notation in which p represents both a predicate and its Gödel code. Both the E and A functions are general recursive, not primitive recursive, since they each express an unbounded search for an arbitrarily-large step count s . With these definitions, we consider a universally-quantified predicate ‘ $\exists x p\langle x, \dots \rangle$ ’ in GA to be a metalogical shorthand for ‘ $E(\ulcorner x \urcorner, \ulcorner p \urcorner, 0)$ ’, while ‘ $\forall x p\langle x, \dots \rangle$ ’ is a metalogical shorthand for ‘ $A(\ulcorner x \urcorner, \ulcorner p \urcorner, 0)$ ’.

At each step s , E invokes the strictly-positive existential quantification function E^+ to test whether a natural number n satisfying the predicate p has been found by step s , terminating with a boolean true result if so. Otherwise, E next invokes the strictly-positive universal quantification function A^+ to test whether it can find within s steps a BGA proof that the predicate p can *never* be satisfied, terminating with a boolean false result if so. If neither of these events occurs, E simply invokes itself recursively with a larger step count $s + 1$, in effect performing an unbounded search for any s that eventually satisfies either the existential quantifier’s positive or negative termination cases.

The function A representing the two-sided universal quantifier works identically, only invoking A^+ in its positive-case termination path and E^+ in its negative-case path.

Because BGA is consistent (Section 3.5), there can be no step count s for which the positive- and negative-case paths of these computations both succeed. If a natural number n exists that can eventually (at some step count s) satisfy the existential quantifier’s positive-case path, for example, then its negative-case path can never terminate regardless of s . Otherwise there would also be a universal negative-case proof that the predicate p yields false given any natural number, and this proof could be instantiated with n to contradict the positive-case existence proof directly.

Since BGA and GA are paracomplete, we fully expect to find cases in which neither the positive- nor negative-case paths of these quantifier computations terminate. We can construct simple examples like the “universal liar” via the definition ‘ $L \equiv \forall x \neg L$ ’, or the “existential truthteller” as ‘ $T \equiv \exists x T$ ’.

4.3 Inductive quantification

As a variation on the quantification rules above in Table 5, we can optionally add, or replace the $\forall I1$ rule with, the following inductive universal quantifier introduction rule:

$$\frac{p\langle 0, \dots \rangle \quad x \mathbf{N}, p\langle x, \dots \rangle \vdash p\langle \mathbf{S}(x), \dots \rangle}{\forall x p\langle x, \dots \rangle} \forall Ind$$

We can similarly modify the A^+ and A definitions above to express a computation that reflectively searches for a BGA proof corresponding to the inductive pattern in this rule: i.e., a BGA proof of the base case ‘ $p\langle 0, \dots \rangle$ ’ together with a BGA proof for the inductive step ‘ $x \mathbf{N}, p\langle x, \dots \rangle \vdash p\langle \mathbf{S}(x), \dots \rangle$ ’.

This variation is interesting because it potentially allows us to remove BGA’s induction rule *Ind* (Table 2), without fundamentally reducing expressiveness, since we can treat both induction and quantification as non-primitive shorthands for computations. The resulting induction-free BGA loosely parallels classical Robinson arithmetic [Robinson, 1950, Mendelson, 2015], which removes induction from Peano arithmetic but remains powerful enough to express arbitrary computation and prove Gödel’s theorems.

5 Mechanically-checked formalization of BGA and (partially) GA in Isabelle/HOL

We have formalized all of BGA and most of full GA with the Isabelle proof assistant, using Isabelle’s higher-order logic HOL as our metalogic [Nipkow et al., 2002]. This section summarizes the status of this formalization. The main purpose of this formalization is to verify as rigorously as reasonably possible that the formal

Cantor pairing and Gödel coding	930
Abstract syntax for logic	1149
Classical logic and arithmetic	1992
Proof and model theory	665
Recursion theory	4544
Domain theory	802
Grounded deduction	1261
Grounded arithmetic	1928
Total	13271

Table 6: Metalogical reasoning framework in Isabelle/HOL: summary of functionality and lines of code

foundation that GA represents is solid. Convenience and richness of expressive power are secondary for the moment, although we informally discuss these considerations later in Section 6.

We formalized GA within a new metalogical reasoning framework we built atop Isabelle/HOL, most of which is used by but not specific to our formalization of GA. Table 6 summarizes the framework’s main functionality areas and the number of HOL code lines that each represent.

The framework first formalizes Cantor’s pairing function $\langle x, y \rangle \equiv (x + 1)(x + y + 1)/2 + y$, defines a Haskell-style type class `coding` to represent HOL types that can be Gödel coded via an injection into the natural numbers, then instantiates this class to assign Gödel codes to various standard HOL types such as booleans, lists, and finite sets.

Instead of defining a HOL datatype specific to GA’s syntax, the framework defines and uses throughout a generic term syntax that supports variables and binding via de Bruijn indices, but is oblivious to the particular set of operators used in a particular modeled language such as GA. We make heavy use of Isabelle’s extensible syntax and its *locale* facility for modular reuse of syntax, definitions, and proofs [Ballarin, 2025]. In combination, these facilities enable most of our metalogical definitions and proofs simply to declare which syntactic constructs they *assume exist* and build on, and to avoid being specific to a particular closed syntax, as they would be if they were written using a HOL datatype defining a particular language syntax such as that of BGA or GA.

For reference and to analyze relationships between different formal systems, our framework formalizes not only GA but also substantial portions of classical logic and arithmetic, including a mostly-complete formalization of primitive-recursive arithmetic (PRA). The framework also formalizes the basics of proof theory and model theory, so that we can Gödel encode and reason about proofs explicitly for reflection, and define semantic models for use in truth preservation proofs.

The framework includes a substantial development of recursion theory, defining primitive-recursive and general-recursive functions via inductive function composition [Kleene, 1952, Mendelson, 2015], Gödel-encodable indices for computable functions, and proofs that many of HOL’s basic natural-number arithmetic, boolean, list, and finite set operations are primitive recursive. This boilerplate constitutes much of the relatively-straightforward but tedious proof infrastructure required for Gödel-style reflective reasoning, which serves as the formal foundation for GA’s computable quantifiers (Section 4).

We initially planned to use a denotational-semantic model in our proof of GA’s consistency, and to this end formalized the basics of Dana Scott’s domain theory [Scott, 1982, Streicher, 2006, Cartwright et al., 2016]. We switched to the operational-semantic approach of Section 3.4 when we found that this approach was simpler and adequate for our purposes. Our formulation of GA’s semantics and inference rules was inspired initially by domain theory, however, and we expect that the denotational-semantic approach would also work as well and remains an interesting alternative to explore in the future.

Building atop the more-generic parts of the framework constituting about 10K lines of HOL code, the

portions specific to BGA and GA currently constitute only around 3K lines. Our formalization of BGA as described in Section 3 is complete, including its operational demantics, and our proof that its inference rules are truth-preserving and hence consistent with respect to this semantics. Our formalization of the computable quantifiers in Section 4 is substantially but not yet fully complete. The main element still missing a complete mechanically-checked proof is the correct correspondence of the quantification inference rules (Table 5) with the reflective computations they represent (Section 4.2.3). The main challenge here, of course, is the substantial reasoning infrastructure required for reflection in general, essentially the same challenge that makes machine-checked proofs of Gödel’s incompleteness theorems nontrivial and rare [Paulson, 2014].

6 Discussion: further steps and implications of grounded reasoning

Since GA in essence contradicts the conventional wisdom that powerful and consistent formal reasoning is infeasible in the presence of unconstrained recursive and traditionally-paradoxical definitions, this paper’s focus has been to lay a minimalistic but solid foundation and establish its correctness and consistency as rigorously as possible. Much more work beyond this paper’s scope will be needed, however, to evolve grounded reasoning into a rich and truly convenient alternative to classical or intuitionistic reasoning. This section discusses potential next steps and forward-looking observations pertinent to this longer-term program. Some of these observations are speculative, and are not intended to constitute formal claims or completed research contributions of this work.

Reviewing note: this section is included to offer a preliminary exploration of potentially-interesting implications of GA and ways it may fit into a longer-term program. While we believe that the content of this section is correct and relevant, we are willing to cut any material in this section that the reviewers find too speculative or otherwise questionable.

6.1 Towards richer grounded type systems

In a grounded deduction system intended for regular use, we would of course like a richer and extensible type system supporting more than the two types in GA (natural numbers and booleans). Adding support for other finite structured types such as tuples, lists, finite sets, and least-fixed-point recursive datatypes appears to be a useful though conceptually-straightforward matter of formal-systems engineering, which we leave for future work.

Beyond ordinary finite types, we may reasonably expect that computably-infinite structures such as streams and other Haskell-style lazy computations and “greatest fixed point” types should be readily compatible with grounded reasoning, though again we leave the details to future work. With computable streams, for example, it should be possible to express and formalize the *computable reals*, or computations generating an infinite series of approximations to real numbers [Weihrauch, 2013].

A more ambitious and uncertain open question, however, is whether some form of mathematical idealization analogous to the law of excluded middle (LEM) may be incorporated into grounded reasoning without inconsistency. Without such an idealization, it appears unlikely that something recognizably like the classical real numbers – which behave like precise “points” rather than the fuzzy ranges that the computable reals represent – will be achievable in grounded reasoning. We see promise in a principle we call *reflective excluded middle* (REM): informally, that while reasoning reflectively as in Section 4, we stipulate that for any well-formed proposition p , either p is provably true in BGA, or else *hypothetically assuming* p leads to contradiction. More formally, REM asserts:

$$\vdash \exists P C(P, \ulcorner T \vdash p \urcorner) \vee C(P, \ulcorner p \vdash F \urcorner) \quad (\text{REM})$$

Without getting into details, it appears that BGA plus REM remains provably consistent, while enabling precise “pointwise” comparisons between real numbers formulated in a natural way for grounded reasoning. We leave full exploration of this and promising related directions to future work.

6.2 Towards convenient automated grounded reasoning systems

Leveraging the Isabelle proof assistant’s already-mature support for multiple diverse logics coexisting atop its minimalistic “Pure” metalogic, we have started to prototype a grounded deduction system atop Isabelle/Pure (as opposed to Isabelle/HOL), intended to be a “working” formal system designed for practical use instead of serving the primary purpose of metalogical analysis. Since this project is only starting, however, we make no claims and cannot yet report significant results from it. Developing grounded reasoning approaches in other proof assistants besides Isabelle would also be interesting and useful future work.

Based on our experience with GA so far, it appears possible to prove not just primitive-recursive but higher-order recursive computations terminating in GA, such as Ackermann’s function [Ackermann, 1928, Kleene, 1952], with merely requires a two-level nested inductive proof. Many of these terminating-computation patterns appear amenable to automation, which we hope to implement in our Isabelle-based working prototype and hope to see in other usability-focused embodiments of grounded reasoning.

Whenever a function to be declared happens to be primitive recursive structurally, for example, we hope and expect that Isabelle’s existing `primrec` construct will be adaptable to discharge the relevant *habeas quid* proof obligations for grounded reasoning automatically. After discharging these proof obligations, grounded reasoning essentially reduces to classical reasoning, thus eliminating the convenience cost of grounded reasoning in these common-case situations.

6.3 Gödel’s incompleteness theorems

While we extensively used Gödel’s tools of reflective reasoning in Section 4, what are we to make of Gödel’s incompleteness theorems themselves in the context of GA?

Informally, Gödel’s first incompleteness theorem states that in any consistent classical formal system F that includes arithmetic – making it powerful enough for reflection – there is a true sentence that F cannot prove true, rendering F syntactically incomplete (unable to prove either the truth or the negation of every well-formed sentence). Most of Gödel’s line of reasoning translates readily into GA, including all the basics of Gödel coding and reflection, with one important exception: the key diagonalization lemma fails to be provable at full strength in GA due to a circular *habeas quid* proof obligation. As a result, Gödel’s exact line of classical reasoning fails to translate into GA. The first incompleteness theorem nevertheless remains true and provable for GA, however – almost trivially, in fact, because we can use the directly-expressible liar sentence ‘ $L \equiv \neg L$ ’ in place of the traditional, tediously-constructed Gödel sentence. The fact that GA is syntactically incomplete is no surprise, however, since GA was designed to be paracomplete, not complete.

Gödel’s second incompleteness theorem states that any consistent classical formal system F that includes arithmetic cannot prove itself consistent. The proof of the second incompleteness theorem in essence relies on reflectively proving that the first incompleteness theorem *is provable* in F , and reasoning that if F could prove itself consistent, then that hypothetically-provable fact together with the first incompleteness theorem would lead to inconsistency in F . If we try to reproduce the second incompleteness theorem in GA, however, we find that the loss of the diagonalization lemma yields a solid roadblock. Thus, the second incompleteness theorem appears not to translate into GA.

Tarski’s closely-related undefinability theorem [Tarski, 1983] informally states that no consistent classical system F can contain its own truth predicate – namely any predicate $T(n)$ such that $T(\ulcorner p \urcorner)$ is true exactly when p is true in F . In essence, a classical system F cannot model its own semantics, as we would need in order for F to prove itself consistent via the standard technique of truth-preservation proofs. Instead, F ’s

semantics can be precisely modeled, sufficient for truth-preservation proofs, only in some strictly-stronger classical system F' . The study of paracomplete logics inspired by Kripke’s theory of truth [Kripke, 1975], however, were motivated precisely by the goal of making it possible for a logic to contain its own truth predicate without causing inconsistency. While the conventional wisdom is that paracomplete logics are intellectually interesting but not usable or powerful enough for practical reasoning [Maudlin, 2006, Field, 2008], GA appears to offer a counterexample to this conventional wisdom by its ability to express directly and reason about arbitrary recursive computations.

Although our mechanically-checked consistency proof of BGA uses Isabelle’s classical HOL as our metalogic (Section 5), we observe informally that all of the reasoning this proof depends on is essentially “ordinary” reasoning about computation, making heavy use of induction and quantifiers – but at no point requiring the use of any particularly exotic or “large” sets, ordinals, or types in HOL. In essence, while performing this consistency proof in HOL we encountered no reasoning steps that would appear *not* readily portable into proofs in the full GA system with computable quantifiers (Section 4). Since the computable quantifiers are just metalogical shorthands that reduce to quantifier-free BGA terms, in effect we see no obvious roadblocks to BGA modeling its own semantics and proving itself consistent. This line of reasoning, if successful, would show that both Gödel’s second incompleteness theorem and Tarski’s undefinability theorem are not only unprovable about, but indeed untrue of, GA. In effect, Hilbert’s second problem [Hilbert, 1900] may be unsolvable due to Gödel only in classical formal systems, but readily solvable in non-classical formal systems like GA. Rigorously checking this line of reasoning is thus an important next step.

6.4 Towards a completeness class of computational formal systems

Would it actually be useful and important if GA could indeed model its own semantics and prove itself consistent? An obvious objection is that in order to perform our truth preservation proof, we effectively had to assume and use all of the key basic logical concepts at the metalogic level in some form in order to prove their correct embodiment in the target logic. If metalogic and target logic are identical, aren’t we essentially guilty of circular reasoning, basically *assuming* that BGA-as-metalogic is correct in order to *prove* that BGA-as-target-logic is correct?

This objection is important, and we see no way around the basic logical concepts being in a sense *self-justifying*: i.e., that we must somehow assume them metalogically in order to justify the same concepts in a target logic. But there are doubtless innumerable contrasting ways to formulate these basic logical concepts, each with distinctive tradeoffs. If a single grounded reasoning system can model its own semantics and prove itself consistent, then there may well be a large class of grounded formal systems, each of which is logically equivalent to and equiconsistent with all the others: that is, a completeness class of formal systems for *reasoning about computation* analogous to the well-established Turing completeness class of computation models. Every further distinct formulation of a similar system that we find equivalent would add to our evidence and certainty that this class is in essence “complete” – even if we see no obvious prospect of *proving* it so definitively. Such a prospect is clearly impossible for classical formal systems due to Gödel and Tarski’s theorems.

Curry’s combinatorial logic [Curry, 1930] and Church’s untyped lambda calculus [Church, 1932], for example, were formal systems intended for reasoning about computation, but both were inconsistent [Kleene and Rosser, 1935]. This discovery motivated Church’s simply-typed lambda calculus [Church, 1941], which achieved consistency by constraining expressible functions to a subset of strictly-terminating computations. GA’s alternate approach to achieving consistency in the presence of unconstrained recursive computation, however, suggests that it might be possible to complete Curry’s and Church’s original program without constraining the functions expressible. Such a grounded system, with computable functions instead of natural numbers as its primitive objects, might well be equivalent to and equiconsistent with GA, coinhabitants of this hypothesized completeness class of grounded formal reasoning systems.

6.5 Reasoning metalogically about ungrounded terms

From the standpoint of a powerful classical metalogic, we can use the operational semantics in Section 3.4 to prove formally that particular GA terms of interest are *ungrounded* [Kripke, 1975], failing to denote any boolean truth value. Consider the liar sentence ‘ $L \equiv \neg L$ ’, for example, and suppose hypothetically that L reduced to either boolean truth value after any particular reduction step count k . Using elimination lemmas readily derivable from the operational semantics, we can show that L must have *already* reduced to some truth value at some strictly-lower step count $k' < k$. But since step counts are natural numbers and cannot decrease without bound, we have used proof by infinite descent to show that our assumption leads to contradiction, so L cannot reduce to any truth value and hence is ungrounded. By the same reasoning we find that the truth-teller sentence ‘ $T \equiv T$ ’ is similarly ungrounded in GA, even though the truth-teller is not classically paradoxical.

As special cases of the observations above, this same reasoning appears to work perfectly well within GA itself: we can prove reflectively in GA-as-metalogic that certain terms of GA-as-target-logic, such as L and T , are ungrounded. While proof by infinite descent is a form of proof by contradiction, which is available only with *habeas quid* preconditions in GA, these preconditions are satisfiable in this case. Testing whether a given GA term reduces to a concrete value in a given number of steps k is primitive recursive, so we can prove that this reduction-test function always terminates, and satisfy in turn the *habeas quid* precondition for proofs by contradiction (including by infinite descent) that this reduction test never succeeds on certain terms for any k . Thus, GA appears to be capable of proving reflectively that the liar L and truth-teller T are ungrounded in GA.

This observation might tempt us to define a function like the following to characterize ungrounded or “gappy” terms [Field, 2008] as those for which there exists no proof either of ‘ t ’ nor of ‘ $\neg t$ ’:

$$G(t) \equiv \neg \exists P (C(P, \vdash t) \vee C(P, \vdash \neg t))$$

Any hopes of this definition being particularly useful, however, are quickly dashed by a variant of the “strengthened liar” problem well-known in the philosophy of logic [Rieger, 2001, Field, 2008, Beall, 2008]:

$$L' \equiv \neg(L' \vee G(\vdash L'))$$

By classical semantic reasoning, it would seem L' must be true if it is either false or ungrounded, but in that case L' must be false, again a self-contradiction. Considering $G(t)$ as a computable function in GA’s semantics, however, we see that while $G(\vdash L)$ and $G(\vdash T)$ terminates (yielding true) on the basic liar and truth-teller sentences, $G(\vdash L')$ again expresses a nonterminating computation and just denotes $\perp$. Thus, $G(t)$ still expresses at best a “gappy” characterization of “gappy” truth. We could formulate higher-order gappiness detectors G' , G'' , etc., but all of these characterizations will still remain unsatisfyingly gappy: the liar paradox may be strengthened without bound.

Despite $G(t)$ being unsatisfactory in this way, however, we find that we can metalogically prove $G(\vdash L')$ to be ungrounded – certainly from a powerful metalogic outside of GA and apparently even reflectively from within GA. We can use a proof by infinite descent similar to those we used above for the basic liar and truth-teller, only strengthening the induction hypothesis to the effect that if either of the terms L' or $G(\vdash L')$ reduce to a truth value at some step count k , then one of those terms must already have reduced to a truth value at some strictly-smaller step count $k' < k$. We can prove a similar metalogical result for any particular strengthening of the liar paradox: in each case, we merely “step back” one metalogical level and use an induction hypothesis strengthened to prove a sufficiently-large set of GA terms all to be ungrounded. Metalogical reflection is powerful.

6.6 Other logical and semantic paradoxes from a grounded perspective

From a perspective of formal logic and semantics, GA is promising because it appears to offer arguably-reasonable resolutions to many well-known logical, semantic, and even set-theoretic paradoxes, not just the Liar and Curry’s paradox discussed earlier. To summarize a few, briefly:

6.6.1 Yablo’s paradox

Yablo’s “paradox without self-reference” [Yablo, 1985, Yablo, 1993] replaces direct circularity with quantification. Suppose we have an infinite series of statements Y_i for each natural number i , each claiming, “all higher-numbered statements are false.” Reasoning classically, if any Y_i is true, then so is Y_{i+1} , but that makes Y_i false. But if Y_i is false for all i , then Y_0 is true.

Leaving aside the debate as to whether Yablo’s paradox truly avoids self-reference [Priest, 1997, Sorensen, 1998, Beall, 2001, Bueno and Colyvan, 2003], the combination of unconstrained recursion together with quantification makes GA perhaps the first realistic formal system in which we can express Yablo’s paradox directly with a valid definition:

$$Y(i) \equiv \forall j (j > i \rightarrow \neg Y(j))$$

Reasoning within GA, we find that to complete a *habeas quid* proof that $Y(i)$ is boolean for any i , we would first have to prove $Y(j)$ boolean for some $j > i$, yielding a bottomless proof obligation. Reasoning metalogically, Y is of course just a non-terminating computation that denotes $\perp$ for any i .

6.6.2 Berry paradox

Informally, Berry’s [Griffin, 2003] or Richard’s [Girard, 2011] paradox of definability asks: what number does the phrase “the smallest natural number not definable in under sixty letters” denote? Whatever that number, the phrase appears to name it in 55 letters. Though informal, this reasoning can be made formally rigorous and used to prove Gödel’s theorems [Chaitin, 1995].

We can similarly express the Berry paradox as a function $B(n)$ in GA, again using Gödel’s reflection tools. B is essentially a computation that searches for and returns the first natural number not definable by a GA term of under n symbols. For some inputs n this function terminates, e.g., $n = 0$. In order for $B(n)$ to terminate with some result r , however, it must first check exhaustively that each $r' < r$ is definable in under n symbols. For interesting values of n , this sub-computation invokes itself recursively with identical arguments, and thus fails to terminate due to infinite recursion. Metalogically, therefore, $B(n)$ again simply denotes $\perp$ in GA for the “paradoxical” values of n .

6.6.3 Banach-Tarski

In ZF set theory with the Axiom of Choice (AC), Banach and Tarski proved the nonintuitive result that a three-dimensional unit ball may be decomposed into a finite number of pieces, then via only volume-preserving rigid transformations, reassembled into two unit balls. This result historically focused a great deal of suspicion on the Axiom of Choice, until measure theory arrived and offered an explanation for the paradox widely deemed satisfactory [Tomkowicz and Wagon, 2016].

Though depending on a notion of grounded sets beyond this paper’s scope, a preliminary grounded analysis intriguingly redirects our attention away from the Axiom of Choice, and instead towards Cantor’s theorem that real numbers are uncountable. Grounded sets satisfy the choice principle inductively without needing an axiom. Examining Cantor’s diagonalization argument, however, which he uses to construct a real number not present in any hypothetically-assumed enumeration of all real numbers, a grounded analysis finds that this construction is just an infinitely-recursive computation akin to that in the Berry paradox, and hence fails to

construct any real number or lead to a grounded proof. Without Cantor’s theorem, we find ourselves missing the derived result that an uncountable set remains nonempty after removing any countable number of elements, an inference used twice in Banach-Tarski. This grounded perspective suggests an interesting question about the historical furor over Banach-Tarski and the Axiom of Choice: was the wrong culprit on trial?

6.7 Diagonalization arguments: the good, the bad, and the ugly

In *Universality and the Liar*, Simmons systematically explores the distinction between “good” and “bad” diagonalization arguments [Simmons, 1993]. A grounded perspective appears to flip several such arguments, including Cantor’s and Gödel’s, to the opposite column of Simmons’ ledger. Is this a tragedy, a weakness of grounded reasoning, as the conventional wisdom suggests? Expressing dissatisfaction with his own closely-related exploration and development of paracomplete propositional logic, for example, Maudlin complains that it “eliminates all of the profound results of metalogic” [Maudlin, 2006, p.133–134]. Or is this an opportunity – immediately, to gain the “freedom of expression” of unconstrained recursive definition, and more long-term and speculatively, to gain more powerful *positive* uses of Gödel’s reflective tools, in place of many of the roadblocks that his theorems erect in classical systems? Only further development and analysis of grounded reasoning can answer this question.

7 Related work

Church’s inconsistent formal system based on his untyped lambda calculus [Church, 1932] launched an early precedent in GA’s general direction by focusing on recursive computation. Church’s system also anticipated GA in certain details, such as by rejecting the law of excluded middle (like Brouwer’s intuitionism [Brouwer, 1907, Heyting, 1975]) while preserving the law of double negation (like GA but unlike intuitionism).

Scott’s domain theory [Scott, 1982], LCF [Scott, 1993, Milner, 1972, Milner, 1976] and PCF [Plotkin, 1977], together with ideas from Kripke’s theory of truth [Kripke, 1975] and the paracomplete logics it inspired [Maudlin, 2006, Field, 2008], contain in scattered form most of the key ideas that inspired GA. Speculating a bit further, perhaps Scott and Kripke might well have formulated a formal system much like GA in the 1970s if they had worked together.

Most proof assistants rely for consistency on stratified type theories in the tradition of Russell [Whitehead and Russell, 2011] and Martin-Löf [Martin-Löf, 1972, Martin-Löf, 1980]. Besides avoiding the known logical paradoxes, these type systems aid automated reasoning by constraining the deduction search space. Dependent type systems as used in proof assistants like Coq/Rocq [Chlipala, 2013], Lean [de Moura and Ullrich, 2021], and Agda [Bove et al., 2009], increase expressiveness by allowing types to depend on computations. Preserving consistency and avoiding Girard’s paradox [Girard, 1972, Hurkens, 1995], however, still requires assigning types to stratified *universes*, which limits expressiveness, motivates numerous variations [Bauer et al., 2020], and complicates desirable features such as polymorphism [Poiret et al., 2025] and metaprogramming [Hu and Pientka, 2025]. By presenting an alternative to stratification for preserving consistency, grounded reasoning as in GA might in the future enable simpler or more flexible dependent type systems, at a cost of weaker and less-familiar deduction rules with *habeas quid* preconditions of course.

A large body of existing work focuses on proving program termination [Colón and Sipma, 2002, Cook et al., 2006, Yao et al., 2024] and non-termination [Gupta et al., 2008, Chatterjee et al., 2021, Raad et al., 2024]. This work generally relies on classical reasoning about computation, of course. While GA introduces a different logical approach to termination proving via its *habeas quid* or dynamic-typing preconditions, nevertheless we hope and expect that most of the existing work on automated or semi-automated termination and non-termination proving should be readily portable into a grounded reasoning context. Working out the details remains for future work, however.

The way in which we satisfy *habeas quid* preconditions or prove termination in GA, essentially by evaluating potentially-abstract or symbolic GA term in reverse (Section 3.3.1), appears conceptually and stylistically reminiscent of symbolic execution [King, 1976, Baldoni et al., 2018, de Boer and Bonsangue, 2021, He et al., 2021]. Like automated termination proving, however, symbolic execution generally focuses on achieving useful results automatically (e.g., finding concrete bugs in deployed software) with manageable time and storage complexity, while managing the exponential state-space explosion problem. While termination-proving in GA is similar in principle to symbolic execution, we make no pretense that GA’s built-in flavor of “symbolic execution” is actually *practical* in terms of time or space complexity; we claim only that GA’s form of “symbolic execution” is *Turing computable*. We hope and expect that practical symbolic-execution techniques could be transplanted into GA, and are thus orthogonal and complementary to GA’s goals, although we again leave further exploration of this relationship to future work.

8 Conclusion and future work

Grounded arithmetic or GA represents a first step towards practical paracomplete formal reasoning about computation. GA allows direct expression of unconstrained recursive definitions, including those leading directly to logical paradox and inconsistency in classical and intuitionistic systems. GA defuses such paradoxes by adding *habeas quid* or dynamic-typing preconditions to key propositional and predicate-logic inference rules. Atop the quantifier-free basic grounded arithmetic (BGA), the quantifiers defining the full first-order grounded predicate logic GA are expressible as ordinary reflective BGA computations. Much work remains both to make grounded reasoning more type-rich and convenient in practical reasoning about real software, and to develop grounded metatheory further. A mechanically-verified proof model of BGA and proof of its consistency, however, suggests that grounded reasoning represents a solid alternative foundation to reasoning about computation.

References

- [Ackermann, 1928] Ackermann, W. (1928). Zum Hilbertschen aufbau der reellen zahlen. *Mathematische Annalen*, 99:118–133.
- [Baldoni et al., 2018] Baldoni, R., Coppa, E., D’elia, D. C., Demetrescu, C., and Finocchi, I. (2018). A survey of symbolic execution techniques. *ACM Computing Surveys (CSUR)*, 51(3):1–39.
- [Ballarin, 2025] Ballarin, C. (2025). Tutorial to locales and locale interpretation.
- [Bauer et al., 2020] Bauer, A., Haselwarter, P. G., and Lumsdaine, P. L. (2020). A general definition of dependent type theories. arXiv preprint 2009.05539.
- [Beall, 2001] Beall, J. (2001). Is Yablo’s paradox non-circular? *Analysis*, 61(3):176–187.
- [Beall, 2008] Beall, J., editor (2008). *Revenge of the Liar: New Essays on the Paradox*. Oxford University Press.
- [Bishop, 1967] Bishop, E. (1967). *Foundations of Constructive Analysis*. Academic Press, New York.
- [Bove et al., 2009] Bove, A., Dybjer, P., and Norell, U. (2009). A brief overview of Agda - a functional language with dependent types. In *22nd International Conference on Theorem Proving in Higher Order Logics (TPHOLs)*, page 73–78. Springer-Verlag.

- [Brouwer, 1907] Brouwer, L. E. J. (1907). *Over de Grondslagen der Wiskunde*. PhD thesis, Universiteit van Amsterdam.
- [Brouwer, 1981] Brouwer, L. E. J. (1981). *Brouwer's Cambridge Lectures on Intuitionism*. Cambridge University Press.
- [Bueno and Colyvan, 2003] Bueno, O. and Colyvan, M. (2003). Paradox without satisfaction. *Analysis*, 63(2):152–156.
- [Button and Walsh, 2018] Button, T. and Walsh, S. (2018). *Philosophy and Model Theory*. Oxford University Press.
- [Cartwright et al., 2016] Cartwright, R., Parsons, R., and AbdelGawad, M. (2016). Domain theory: An introduction. arXiv preprint 1605.05858.
- [Chaitin, 1995] Chaitin, G. J. (1995). The Berry paradox. *Complexity*, 1(1):26–30.
- [Chatterjee et al., 2021] Chatterjee, K., Goharshady, E. K., Novotný, P., and Đorđe Žikelić (2021). Proving non-termination by program reversal. In *42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, pages 1033–1048.
- [Chlipala, 2013] Chlipala, A. (2013). *Certified Programming with Dependent Types: A Pragmatic Introduction to the Coq Proof Assistant*. The MIT Press.
- [Church, 1932] Church, A. (1932). A set of postulates for the foundation of logic. *Annals of Mathematics*, 33(2):346–366.
- [Church, 1941] Church, A. (1941). *The Calculi of Lambda-Conversion*. Number 6 in Annals of Mathematics Studies. Princeton University Press.
- [Colón and Sipma, 2002] Colón, M. A. and Sipma, H. B. (2002). Practical methods for proving program termination. In *International Conference on Computer Aided Verification*, pages 442–454.
- [Cook et al., 2006] Cook, B., Podelski, A., and Rybalchenko, A. (2006). Termination proofs for systems code. In *27th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*.
- [Curry, 1930] Curry, H. B. (1930). Grundlagen der kombinatorischen logik. *American Journal of Mathematics*, 52(3):509–536.
- [de Boer and Bonsangue, 2021] de Boer, F. S. and Bonsangue, M. (2021). Symbolic execution formally explained. *Formal Aspects of Computing*, 33:617–636.
- [de Moura and Ullrich, 2021] de Moura, L. and Ullrich, S. (2021). The Lean 4 theorem prover and programming language. In *28th International Conference on Automated Deduction (CADE)*, pages 625–635.
- [Field, 2008] Field, H. (2008). *Saving Truth From Paradox*. Oxford University Press.
- [Girard, 1972] Girard, J.-Y. (1972). *Interprétation fonctionnelle et élimination des coupures dans l'arithmétique d'ordre supérieur*. PhD thesis.
- [Girard, 2011] Girard, J.-Y. (2011). *The Blind Spot: Lectures on Logic*. European Mathematical Society.
- [Griffin, 2003] Griffin, N. (2003). *The Cambridge Companion to Bertrand Russell*. Cambridge Companions to Philosophy. Cambridge University Press.

- [Gupta et al., 2008] Gupta, A., Henzinger, T. A., Majumdar, R., Rybalchenko, A., and Xu, R.-G. (2008). Proving non-termination. In *35th annual ACM SIGPLAN-SIGACT symposium on Principles of programming languages (POPL)*, pages 147–158.
- [He et al., 2021] He, J., Sivanrupan, G., Tsankov, P., and Vechev, M. (2021). Learning to explore paths for symbolic execution. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2526–2540.
- [Heyting, 1971] Heyting, A. (1971). *Intuitionism: An introduction*. Studies in logic and the foundations of mathematics. North-Holland.
- [Heyting, 1975] Heyting, A., editor (1975). *Philosophy and Foundations of Mathematics*, volume 1 of *L. E. J. Brouwer Collected Works*. North-Holland.
- [Hilbert, 1900] Hilbert, D. (1900). Mathematical problems. Lecture delivered before the International Congress of Mathematicians at Paris.
- [Hoare, 1969] Hoare, C. (1969). An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580.
- [Howard, 1980] Howard, W. A. (1980). The formulae-as-types notion of construction. In *To H.B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, pages 479–490. Academic Press.
- [Hu and Pientka, 2025] Hu, J. Z. S. and Pientka, B. (2025). A dependent type theory for meta-programming with intensional analysis. In *Proceedings of the ACM on Programming Languages*, volume 9, pages 416–445.
- [Hurkens, 1995] Hurkens, A. J. C. (1995). A simplification of Girard’s paradox. In *International Conference on Typed Lambda Calculi and Applications*, pages 266–278. Springer Berlin Heidelberg.
- [King, 1976] King, J. C. (1976). Symbolic execution and program testing. *Communications of the ACM*, 19(7):385–394.
- [Kleene, 1943] Kleene, S. C. (1943). Recursive predicates and quantifiers. *Transactions of the American Mathematical Society*, 53(1):41–73.
- [Kleene, 1952] Kleene, S. C. (1952). *Introduction to Metamathematics*. North-Holland.
- [Kleene and Rosser, 1935] Kleene, S. C. and Rosser, J. B. (1935). The inconsistency of certain formal logics. *Annals of Mathematics*, 36(3):630–636.
- [Klein et al., 2014] Klein, G., Andronick, J., Elphinstone, K., Murray, T., Sewell, T., Kolanski, R., and Heiser, G. (2014). Comprehensive formal verification of an OS microkernel. *ACM Transactions on Computer Systems (TOCS)*, 32(1):1–70.
- [Kripke, 1975] Kripke, S. (1975). Outline of a theory of truth. *The Journal of Philosophy*, 72(19):690–716.
- [Kunčar and Popescu, 2018] Kunčar, O. and Popescu, A. (2018). A consistent foundation for Isabelle/HOL. *Journal of Automated Reasoning*, 62:531–555.
- [Leroy et al., 2016] Leroy, X., Blazy, S., Kästner, D., Schommer, B., Pister, M., and Ferdinand, C. (2016). CompCert – a formally verified optimizing compiler. In *Embedded Real Time Software and Systems (ERTS)*.

- [Martin-Löf, 1972] Martin-Löf, P. (1972). An intuitionistic theory of types. Technical report, University of Stockholm.
- [Martin-Löf, 1980] Martin-Löf, P. (1980). Intuitionistic type theory. Notes by Giovanni Sambin of a series of lectures given in Padua, June 1980.
- [Maudlin, 2006] Maudlin, T. (2006). *Truth and Paradox*. Clarendon Press.
- [Mendelson, 2015] Mendelson, E. (2015). *Introduction to Mathematical Logic*. Routledge, 6th edition.
- [Milner, 1972] Milner, R. (1972). Implementation and applications of Scott’s logic for computable functions. 7(1):1–6.
- [Milner, 1976] Milner, R. (1976). Models of LCF. Technical report, University of Edinburgh. Mathematical Centre Tracts.
- [Milner, 1999] Milner, R. (1999). *Communicating and Mobile Systems: The Pi Calculus*. Cambridge University Press.
- [Moschovakis, 2016] Moschovakis, Y. N. (2016). Hyperarithmetical sets. In Omodeo, E. G. and Policriti, A., editors, *Martin Davis on Computability, Computational Logic, and Mathematical Foundations*, volume 10 of *Outstanding Contributions to Logic*, pages 107–149. Springer.
- [Mostowski, 1947] Mostowski, A. (1947). On definable sets of positive integers. *Fundamenta Mathematicae*, 34:81–112.
- [Nipkow et al., 2002] Nipkow, T., Paulson, L. C., and Wenzel, M. (2002). *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*. Springer. Lecture Notes in Computer Science, 2283.
- [Norell, 2007] Norell, U. (2007). *Towards a practical programming language based on dependent type theory*. PhD thesis.
- [Paulson, 2014] Paulson, L. C. (2014). A machine-assisted proof of Gödel’s incompleteness theorems for the theory of hereditarily finite sets. *The Review of Symbolic Logic*, 7(3):484–498.
- [Plotkin, 1977] Plotkin, G. (1977). LCF considered as a programming language. *Theoretical Computer Science*, (5):223–255.
- [Poiret et al., 2025] Poiret, J., Gilbert, G., Maillard, K., Pédrot, P.-M., Sozeau, M., Tabareau, N., and Éric Tanter (2025). All your base are belong to U^s : Sort polymorphism for proof assistants. In *Proceedings of the ACM on Programming Languages*, volume 9, pages 2253–2281.
- [Priest, 1997] Priest, G. (1997). Yablo’s paradox. *Analysis*, 57(4):236–242.
- [Priest, 2006] Priest, G. (2006). *In Contradiction: A Study of the Transconsistent*. Clarendon Press, 2nd edition.
- [Quine, 1982] Quine, W. V. O. (1982). *Mathematical Logic: Revised Edition*. Harvard University Press.
- [Raad et al., 2024] Raad, A., Vanegue, J., and O’Hearn, P. (2024). Non-termination proving at scale. In *Proceedings of the ACM on Programming Languages*, volume 8, pages 246–274.
- [Reynolds, 2002] Reynolds, J. (2002). Separation logic: a logic for shared mutable data structures. In *17th Annual IEEE Symposium on Logic in Computer Science*.

- [Rieger, 2001] Rieger, A. (2001). The liar, the strengthened liar, and bivalence. *Erkenntnis*, 54:195–203.
- [Robinson, 1950] Robinson, R. M. (1950). An essentially undecidable axiom system. In *Proceedings of the International Congress of Mathematics*, pages 729–730.
- [Scott, 1982] Scott, D. S. (1982). Domains for denotational semantics. In *International Colloquium on Automata, Languages and Programming (ICALP)*.
- [Scott, 1993] Scott, D. S. (1993). A type-theoretical alternative ISWIM, CUCH, OWHY. *Theoretical Computer Science*, 121:411–440.
- [Simmons, 1993] Simmons, K. (1993). *Universality and the Liar: An Essay on Truth and the Diagonal Argument*. Cambridge University Press.
- [Skolem, 1923] Skolem, T. (1923). Begründung der elementaren arithmetik durch die rekurrierende denkweise ohne anwendung scheinbarer veränderlichen mit unendlichem ausdehnungsbereich. *Videnskapselskapets skrifter*, I. Matematisk-naturvidenskabelig klasse(6).
- [Sorensen, 1998] Sorensen, R. A. (1998). Yablo’s paradox and kindred infinite liars. *Mind*, 107(425):137–155.
- [Streicher, 2006] Streicher, T. (2006). *Domain-Theoretic Foundations of Functional Programming*. World Scientific.
- [Tarski, 1983] Tarski, A. (1983). The concept of truth in formalized languages. In Corcoran, J., editor, *Logic, Semantics, Metamathematics: Papers from 1923 to 1938*. Hackett Publishing Company.
- [Tomkowicz and Wagon, 2016] Tomkowicz, G. and Wagon, S. (2016). *The Banach–Tarski Paradox*. Number 163 in *Encyclopedia of Mathematics and its Applications*. Cambridge University Press, 2nd edition.
- [van Heijenoort, 2002] van Heijenoort, J., editor (2002). *From Frege to Gödel: A Source Book in Mathematical Logic, 1879–1931*. Source Books in History of Sciences. Fourth printing edition.
- [Weihrauch, 2013] Weihrauch, K. (2013). *Computable Analysis: An Introduction*. Texts in Theoretical Computer Science. Springer.
- [Whitehead and Russell, 2011] Whitehead, A. N. and Russell, B. (2011). *Principia Mathematica*, volume 1. Rough Draft Printing.
- [Winskel, 1993] Winskel, G. (1993). *Formal Semantics of Programming Languages*. The MIT Press.
- [Yablo, 1985] Yablo, S. (1985). Truth and reflection. *Journal of Philosophical Logic*, 14:297–349.
- [Yablo, 1993] Yablo, S. (1993). Paradox without self-reference. *Analysis*, 53(4):251–252.
- [Yao et al., 2024] Yao, J., Tao, R., Gu, R., and Nieh, J. (2024). Mostly automated verification of liveness properties for distributed protocols with ranking functions. In *Proceedings of the ACM on Programming Languages*, volume 8, pages 1028–1059.

A Detailed derivations

This appendix contains more details on derivations that are briefly mentioned in the main paper.

A.1 Proof by contradiction in GA

Theorem A.1. *From the base rules of BGA in table Table 2, we can derive the following grounded deduction rule for refutation by contradiction:*

$$\frac{p \text{ B} \quad p \vdash q \quad p \vdash \neg q}{\neg p}$$

Proof. Because ‘ $p \text{ B}$ ’ is shorthand for ‘ $p \vee \neg p$ ’ from Table 1b, the derivation uses $\vee E1$ to perform case analysis on the typing judgment. In the case that ‘ p ’ holds, we can obtain both ‘ q ’ and ‘ $\neg q$ ’ which allows us to apply $\neg E$ to conclude anything, in this case ‘ $\neg p$.’ In the case that ‘ $\neg p$ ’ holds, the conclusion is immediate. Because we can conclude ‘ $\neg p$ ’ in both cases, we obtain the unconditional conclusion. $\square$

Theorem A.2. *From the base rules of BGA, we can derive this rule for proof by contradiction:*

$$\frac{p \text{ B} \quad \neg p \vdash q \quad \neg p \vdash \neg q}{p}$$

Proof. As above, we proceed by case analysis on the judgment ‘ $p \text{ B}$.’ In the case that ‘ p ’ holds, the conclusion is immediate. In the case that ‘ $\neg p$ ’ holds, we again obtain both ‘ q ’ and ‘ $\neg q$ ’ so we can use $\neg E$ to conclude ‘ p .’ In any case, we can obtain ‘ p ,’ so $\vee E1$ allows us to unconditionally conclude ‘ p .’ $\square$

A.2 Deriving conjunction from disjunction

Theorem A.3. *From the base rules of BGA, we can derive the conjunction introduction rule:*

$$\frac{p \quad q}{p \wedge q} \wedge I$$

Proof. Suppose we already have proofs of ‘ p ’ and ‘ q .’ Unfolding the shorthand for conjunction, we wish to prove ‘ $\neg(\neg p \vee \neg q)$.’ We can use double negation introduction to obtain ‘ $\neg(\neg p)$ ’ and ‘ $\neg(\neg q)$,’ which allows us to apply $\vee I3$ to obtain ‘ $\neg(\neg p \vee \neg q)$ ’ as desired. $\square$

Theorem A.4. *From the base rules of BGA, we can derive the conjunction elimination rules:*

$$\frac{p \wedge q}{p} \wedge E1 \quad \frac{p \wedge q}{q} \wedge E2$$

Proof. To derive $\wedge E1$, suppose we already have ‘ $\neg(\neg p \vee \neg q)$.’ We can use $\vee E2$ to obtain ‘ $\neg\neg p$ ’ followed by double negation elimination to obtain ‘ p .’ The proof of $\wedge E2$ is analogous. $\square$

A.3 Implication and biconditional in GA

Theorem A.5. *From the base rules of BGA, we can derive grounded implication introduction:*

$$\frac{p \text{ B} \quad p \vdash q}{p \rightarrow q} \rightarrow I$$

Proof. The introduction rule adds the *habeas quid* precondition to the classical rule, allowing us to perform a case analysis on the judgment ‘ $p \mathbf{B}$ ’ to conclude ‘ $p \rightarrow q$ ’ from a proof that ‘ $p \vdash q$.’ In the case that ‘ p ’ holds, we use the hypothetical premise ‘ $p \vdash q$ ’ to conclude ‘ q ’ and therefore ‘ $\neg p \vee q$.’ In the case that ‘ $\neg p$ ’ holds, the conclusion is immediate. In either case, we have ‘ $\neg p \vee q$ ’ and therefore, using the shorthand for implication from Table 1b, ‘ $p \rightarrow q$.’ $\square$

Theorem A.6. *From the base rules of BGA, we can derive implication elimination:*

$$\frac{p \rightarrow q \quad p}{q} \rightarrow E$$

Proof. Unfolding the shorthand for implication in Table 1b, we can perform case elimination on ‘ $\neg p \vee q$.’ In the case that ‘ $\neg p$,’ we have a contradiction because we also have ‘ p ’ from the other premise. We can then use $\neg E$ to conclude ‘ q .’ In the other case, the conclusion ‘ q ’ is immediate. $\square$

The biconditional rules now follow quickly from the implication rules.

Theorem A.7. *From the base rules of BGA, we can derive biconditional introduction:*

$$\frac{p \mathbf{B} \quad q \mathbf{B} \quad p \vdash q \quad q \vdash p}{p \leftrightarrow q} \leftrightarrow I$$

Proof. We can use $\rightarrow I$ to obtain ‘ $p \rightarrow q$ ’ from ‘ $p \mathbf{B}$ ’ and ‘ $p \vdash q$ ’ and, similarly, ‘ $q \rightarrow p$ ’ from ‘ $q \mathbf{B}$ ’ and ‘ $q \vdash p$.’ We can then use the derived rule $\wedge I$ to obtain ‘ $(p \rightarrow q) \wedge (q \rightarrow p)$,’ which is equivalent to ‘ $p \leftrightarrow q$.’ $\square$

Theorem A.8. *From the base rules of BGA, we can derive the biconditional elimination rules:*

$$\frac{p \leftrightarrow q \quad p}{q} \leftrightarrow E1 \quad \frac{p \leftrightarrow q \quad q}{p} \leftrightarrow E2$$

Proof. To derive $\leftrightarrow E1$, we use conjunction elimination on ‘ $p \leftrightarrow q$ ’ to obtain ‘ $p \rightarrow q$,’ then apply $\rightarrow E$. The derivation is the same for $\leftrightarrow E2$, only using the other conjunction elimination rule. $\square$

A.4 Transitivity of equality

Theorem A.9. *From the base rules of BGA, we can derive*

$$\frac{a = b \quad b = c}{a = c} = T$$

Proof. Suppose we already have proofs that ‘ $a = b$ ’ and ‘ $b = c$.’ We apply $=E$, instantiating ‘ p ’ as the predicate ‘ $p(x) \equiv a = x$.’ We know that ‘ $p(b) \equiv a = b$ ’ holds, and since ‘ $b = c$,’ we can also substitute ‘ c ’ for ‘ b ’ using the $=E$ rule to obtain the conclusion ‘ $a = c$ ’ as desired. $\square$

A.5 Termination proof examples

We now present some explicit termination proofs, including a full termination proof of `even`, as referenced in Section 3.3.2, and one for multiplication. Termination in GA is implied by the judgment ‘ $a \mathbf{N}$ ’, which is interpreted as ‘ a ’ being a GA term that corresponds to a terminating computation evaluating to a natural number. The simplest case occurs when the expression ‘ a ’ is already a natural number. The oddity of a “terminating” natural number can be thought of as an explicit termination proof of the successive applications of the successor function ‘ $\mathbf{S}$ ’. We thus start with proving termination of the natural numbers 0 and 1 using the GA inference rules.

Theorem A.10. *Zero is a natural number.*

$$\vdash 0 \mathbf{N}$$

Proof. This follows immediately from the inference rule $0I$.

$$\frac{}{\vdash 0 \mathbf{N}} 0I$$

□

Theorem A.11. *One is a natural number.*

$$\vdash \underline{1} \mathbf{N}$$

Proof. Since $\underline{1}$ is a metalogical definition that “converts” a number into its GA term representation, we use the converted form ‘ $\mathbf{S}(0)$ ’ directly.

$$\frac{\frac{}{\vdash 0 \mathbf{N}} 0I}{\vdash \mathbf{S}(0) \mathbf{N}} \mathbf{STIE}$$

□

Let us now turn to a more interesting termination proof. Consider the following recursive definition of addition in GA:

$$\text{add}(x, y) \equiv y = 0 ? x : \mathbf{S}(\text{add}(x, \mathbf{P}(y)))$$

Theorem A.12. *The term ‘ $\text{add}(x, y)$ ’ terminates for any natural numbers ‘ x ’ and ‘ y ’.*

$$x \mathbf{N}, y \mathbf{N} \vdash \text{add}(x, y) \mathbf{N}$$

Proof. By induction on the second argument using the Ind rule. The first two premises correspond to the base case and induction step respectively and their derivation trees are shown separately.

$$\frac{\frac{(1)}{x \mathbf{N}, y \mathbf{N} \vdash \text{add}(x, 0) \mathbf{N}} \quad \frac{(2)}{x \mathbf{N}, y \mathbf{N}, a \mathbf{N}, \text{add}(x, a) \mathbf{N} \vdash \text{add}(x, \mathbf{S}(a)) \mathbf{N}} \quad \frac{x \mathbf{N}, y \mathbf{N} \vdash y \mathbf{N}}{x \mathbf{N}, y \mathbf{N} \vdash \text{add}(x, y) \mathbf{N}} H}{x \mathbf{N}, y \mathbf{N} \vdash \text{add}(x, y) \mathbf{N}} Ind$$

The base case subtree (1):

$$\frac{\frac{\frac{\frac{}{x \mathbf{N}, y \mathbf{N} \vdash 0 = 0} 0I}{x \mathbf{N}, y \mathbf{N} \vdash 0 = 0 ? x : \mathbf{S}(\text{add}(x, \mathbf{P}(0)))} ?II}{x \mathbf{N}, y \mathbf{N} \vdash \text{add}(x, 0) = 0} \equiv IE}{\frac{x \mathbf{N}, y \mathbf{N} \vdash 0 = \text{add}(x, 0)}{x \mathbf{N}, y \mathbf{N} \vdash 0 = \text{add}(x, 0)} = S} \quad \frac{x \mathbf{N}, y \mathbf{N} \vdash 0 \mathbf{N}}{x \mathbf{N}, y \mathbf{N} \vdash \text{add}(x, 0) \mathbf{N}} 0I}{x \mathbf{N}, y \mathbf{N} \vdash \text{add}(x, 0) \mathbf{N}} = E$$

where ‘ $0 = 0$ ’ is resolved by $0I$ since ‘ $0 \mathbf{N}$ ’ is just shorthand for ‘ $0 = 0$.’

The induction step subtree (2):

$$\frac{\frac{\frac{}{\Gamma_0 \vdash 0 \mathbf{N}} 0I}{\Gamma_0 \vdash \mathbf{S}(a) = 0 \mathbf{B}} \quad \frac{\frac{\frac{}{\Gamma_0 \vdash a \mathbf{N}} H}{\Gamma_0 \vdash \mathbf{S}(a) \mathbf{N}} \mathbf{STIE}}{\Gamma_0 \vdash \mathbf{S}(a) = 0 \mathbf{B}} = TI \quad \frac{\frac{\frac{}{\Gamma_0 \vdash x \mathbf{N}} H}{\Gamma_0 \vdash \mathbf{S}(\text{add}(x, \mathbf{P}(\mathbf{S}(a)))) \mathbf{N}} (3)}{\Gamma_0 \vdash \mathbf{S}(a) = 0 ? x : \mathbf{S}(\text{add}(x, \mathbf{P}(\mathbf{S}(a)))) \mathbf{N}} ? TI}{\Gamma_0 \vdash \text{add}(x, \mathbf{S}(a)) \mathbf{N}} \equiv IE$$

where $\Gamma_0 = \{x \mathbf{N}, y \mathbf{N}, a \mathbf{N}, \text{add}(x, a) \mathbf{N}\}$ for brevity.

Subtree (3):

$$\frac{\frac{\frac{\Gamma_0 \vdash a \mathbf{N}}{H} \text{P=I2}}{\Gamma_0 \vdash \mathbf{P}(\mathbf{S}(a)) = a} \text{=S} \quad \frac{\Gamma_0 \vdash \text{add}(x, a) \mathbf{N}}{H}}{\frac{\Gamma_0 \vdash \text{add}(x, \mathbf{P}(\mathbf{S}(a))) \mathbf{N}}{\Gamma_0 \vdash \mathbf{S}(\text{add}(x, \mathbf{P}(\mathbf{S}(a)))) \mathbf{N}} \text{STIE}} \text{=E}$$

where $\Gamma_0 = \{x \mathbf{N}, y \mathbf{N}, a \mathbf{N}, \text{add}(x, a) \mathbf{N}\}$ for brevity. □

Let us now do the analogous termination proof for ‘ $\text{sub}(x, y)$.’ Consider the following recursive definition of subtraction in GA:

$$\text{sub}(x, y) \equiv y = 0 ? x : \mathbf{P}(\text{sub}(x, \mathbf{P}(y)))$$

Theorem A.13. *The term ‘ $\text{sub}(x, y)$ ’ terminates for any natural numbers ‘ x ’ and ‘ y ’.*

$$x \mathbf{N}, y \mathbf{N} \vdash \text{sub}(x, y) \mathbf{N}$$

Proof. The proof is almost exactly the same as the one for Theorem A.12 due to the definition of sub and add differing in only one symbol. It is still spelled out in full rigor for completeness sake. By induction on the second argument using the *Ind* rule. The first two premises correspond to the base case and induction step respectively and their derivation trees are shown separately.

$$\frac{\frac{(1)}{x \mathbf{N}, y \mathbf{N} \vdash \text{sub}(x, 0) \mathbf{N}} \quad \frac{(2)}{x \mathbf{N}, y \mathbf{N}, a \mathbf{N}, \text{sub}(x, a) \mathbf{N} \vdash \text{sub}(x, \mathbf{S}(a)) \mathbf{N}} \quad \frac{x \mathbf{N}, y \mathbf{N} \vdash y \mathbf{N}}{H}}{x \mathbf{N}, y \mathbf{N} \vdash \text{sub}(x, y) \mathbf{N}} \text{Ind}$$

The base case subtree (1):

$$\frac{\frac{\frac{x \mathbf{N}, y \mathbf{N} \vdash 0 = 0}{0I} \quad \frac{x \mathbf{N}, y \mathbf{N} \vdash x \mathbf{N}}{H}}{x \mathbf{N}, y \mathbf{N} \vdash 0 = 0 ? x : \mathbf{P}(\text{sub}(x, \mathbf{P}(0)))} ?II}{\frac{x \mathbf{N}, y \mathbf{N} \vdash \text{sub}(x, 0) = 0}{x \mathbf{N}, y \mathbf{N} \vdash 0 = \text{sub}(x, 0)} \text{=S} \quad \frac{x \mathbf{N}, y \mathbf{N} \vdash 0 \mathbf{N}}{0I}}{x \mathbf{N}, y \mathbf{N} \vdash \text{sub}(x, 0) \mathbf{N}} \text{=E}$$

where ‘ $0 = 0$ ’ is resolved by *0I* since ‘ $0 \mathbf{N}$ ’ is just shorthand for ‘ $0 = 0$.’

The induction step subtree (2):

$$\frac{\frac{\Gamma_0 \vdash 0 \mathbf{N}}{0I} \quad \frac{\frac{\Gamma_0 \vdash a \mathbf{N}}{H} \text{STIE}}{\Gamma_0 \vdash \mathbf{S}(a) = 0 \mathbf{B}} \text{=TI} \quad \frac{\Gamma_0 \vdash x \mathbf{N}}{H} \quad \frac{(3)}{\Gamma_0 \vdash \mathbf{P}(\text{sub}(x, \mathbf{P}(\mathbf{S}(a)))) \mathbf{N}}}{\frac{\Gamma_0 \vdash \mathbf{S}(a) = 0 ? x : \mathbf{P}(\text{sub}(x, \mathbf{P}(\mathbf{S}(a)))) \mathbf{N}}{\Gamma_0 \vdash \text{sub}(x, \mathbf{S}(a)) \mathbf{N}} \text{=IE}} ?TI$$

where $\Gamma_0 = \{x \mathbf{N}, y \mathbf{N}, a \mathbf{N}, \text{sub}(x, a) \mathbf{N}\}$ for brevity.

Subtree (3):

$$\frac{\frac{\frac{\overline{\Gamma_0 \vdash a \mathbf{N}} H}{\Gamma_0 \vdash \mathbf{P}(\mathbf{S}(a)) = a} \mathbf{P} = I2}{\Gamma_0 \vdash a = \mathbf{P}(\mathbf{S}(a))} = S \quad \frac{\overline{\Gamma_0 \vdash \text{sub}(x, a) \mathbf{N}} H}{\Gamma_0 \vdash \text{sub}(x, \mathbf{P}(\mathbf{S}(a))) \mathbf{N}} = E}{\Gamma_0 \vdash \mathbf{P}(\text{sub}(x, \mathbf{P}(\mathbf{S}(a)))) \mathbf{N}} \mathbf{P} TIE$$

where $\Gamma_0 = \{x \mathbf{N}, y \mathbf{N}, a \mathbf{N}, \text{sub}(x, a) \mathbf{N}\}$ for brevity. □

Let us now define **even** using **sub**:

$$\text{even}(n) \equiv n = 0 ? 1 : \text{sub}(1, \text{even}(\mathbf{P}(n)))$$

Theorem A.14. *The term ‘even(n)’ terminates for any natural number ‘n’.*

$$n \mathbf{N} \vdash \text{even}(n) \mathbf{N}$$

Proof. By induction on n using the *Ind* rule. The first two premises correspond to the base case and induction step respectively and their derivation trees are shown separately.

$$\frac{\frac{(1)}{n \mathbf{N} \vdash \text{even}(0) \mathbf{N}} \quad \frac{(2)}{n \mathbf{N}, a \mathbf{N}, \text{even}(a) \mathbf{N} \vdash \text{even}(\mathbf{S}(a)) \mathbf{N}} \quad \frac{n \mathbf{N} \vdash n \mathbf{N}}{n \mathbf{N} \vdash \text{even}(n) \mathbf{N}} H}{Ind}$$

The base case subtree (1):

$$\frac{\frac{\frac{\overline{n \mathbf{N} \vdash 0 = 0} 0I}{n \mathbf{N} \vdash 0 = 0 ? 1 : \text{sub}(1, \text{even}(\mathbf{P}(0)))} ? I1}{n \mathbf{N} \vdash \text{even}(0) = 0} \equiv IE \quad \frac{\overline{n \mathbf{N} \vdash 0 \mathbf{N}} 0I}{n \mathbf{N} \vdash 0 \mathbf{N}} = E}{\frac{n \mathbf{N} \vdash \text{even}(0) = 0}{n \mathbf{N} \vdash 0 = \text{even}(0)} = S \quad \overline{n \mathbf{N} \vdash 0 \mathbf{N}} 0I}{n \mathbf{N} \vdash \text{even}(0) \mathbf{N}} = E$$

where ‘ $0 = 0$ ’ is resolved by *0I* since ‘ $0 \mathbf{N}$ ’ is just shorthand for ‘ $0 = 0$.’

The induction step subtree (2):

$$\frac{\frac{\overline{\Gamma_0 \vdash 0 \mathbf{N}} 0I \quad \frac{\overline{\Gamma_0 \vdash a \mathbf{N}} H}{\Gamma_0 \vdash \mathbf{S}(a) \mathbf{N}} \mathbf{S} TIE}{\Gamma_0 \vdash \mathbf{S}(a) = 0 \mathbf{B}} = TI \quad \frac{\overline{\Gamma_0 \vdash x \mathbf{N}} H \quad \frac{(3)}{\Gamma_0 \vdash \text{sub}(1, \text{even}(\mathbf{P}(\mathbf{S}(a)))) \mathbf{N}}}{\Gamma_0 \vdash \mathbf{S}(a) = 0 ? 1 : \text{sub}(1, \text{even}(\mathbf{P}(\mathbf{S}(a)))) \mathbf{N}} ? TI}{\Gamma_0 \vdash \text{even}(\mathbf{S}(a)) \mathbf{N}} \equiv IE$$

Where $\Gamma_0 = \{n \mathbf{N}, a \mathbf{N}, \text{even}(a) \mathbf{N}\}$ for brevity.

Subtree (3):

$$\begin{array}{c}
\frac{\Gamma_0 \vdash a \mathbf{N} \quad H}{\Gamma_0 \vdash \mathbf{P}(\mathbf{S}(a)) = a} \mathbf{P}=I2 \\
\frac{\Gamma_0 \vdash a = \mathbf{P}(\mathbf{S}(a))}{\Gamma_0 \vdash a = \mathbf{P}(\mathbf{S}(a))} =S \\
\frac{\Gamma_0 \vdash 1 \mathbf{N} \quad \text{Theorem A.11} \quad \frac{\Gamma_0 \vdash a = \mathbf{P}(\mathbf{S}(a)) \quad \Gamma_0 \vdash \mathbf{even}(a) \mathbf{N}}{\Gamma_0 \vdash \mathbf{even}(\mathbf{P}(\mathbf{S}(a))) \mathbf{N}} =E}{\Gamma_0 \vdash \mathbf{sub}(1, \mathbf{even}(\mathbf{P}(\mathbf{S}(a)))) \mathbf{N}} \text{Theorem A.13}
\end{array}$$

Where $\Gamma_0 = \{n \mathbf{N}, a \mathbf{N}, \mathbf{even}(a) \mathbf{N}\}$ for brevity. □

Let us now define **mult**. Consider the following recursive definition of multiplication in GA:

$$\mathbf{mult}(x, y) \equiv y = 0 ? 0 : \mathbf{add}(x, \mathbf{mult}(x, \mathbf{P}(y)))$$

Theorem A.15. *The term ‘ $\mathbf{mult}(x, y)$ ’ terminates for any natural numbers ‘ x ’ and ‘ y ’.*

$$x \mathbf{N}, y \mathbf{N} \vdash \mathbf{mult}(x, y) \mathbf{N}$$

Proof. By induction on the second argument using the *Ind* rule. The first two premises correspond to the base case and induction step respectively and their derivation trees are shown separately.

$$\begin{array}{c}
(1) \quad \frac{}{x \mathbf{N}, y \mathbf{N} \vdash \mathbf{mult}(x, 0) \mathbf{N}} \quad (2) \quad \frac{}{x \mathbf{N}, y \mathbf{N}, a \mathbf{N}, \mathbf{mult}(x, a) \mathbf{N} \vdash \mathbf{mult}(x, \mathbf{S}(a)) \mathbf{N}} \quad \frac{}{x \mathbf{N}, y \mathbf{N} \vdash y \mathbf{N}} H \\
\hline
x \mathbf{N}, y \mathbf{N} \vdash \mathbf{mult}(x, y) \mathbf{N} \quad \text{Ind}
\end{array}$$

The base case subtree (1):

$$\begin{array}{c}
\frac{\frac{}{x \mathbf{N}, y \mathbf{N} \vdash 0 = 0} 0I \quad \frac{}{x \mathbf{N}, y \mathbf{N} \vdash 0 \mathbf{N}} 0I}{x \mathbf{N}, y \mathbf{N} \vdash 0 = 0 ? 0 : \mathbf{add}(x, \mathbf{mult}(x, \mathbf{P}(0)))} ?I1 \\
\hline
\frac{x \mathbf{N}, y \mathbf{N} \vdash \mathbf{mult}(x, 0) = 0}{x \mathbf{N}, y \mathbf{N} \vdash 0 = \mathbf{mult}(x, 0)} =S \quad \frac{}{x \mathbf{N}, y \mathbf{N} \vdash 0 \mathbf{N}} 0I \\
\hline
x \mathbf{N}, y \mathbf{N} \vdash \mathbf{mult}(x, 0) \mathbf{N} \quad =E
\end{array}$$

Where ‘ $0 = 0$ ’ is resolved by *0I* since ‘ $0 \mathbf{N}$ ’ is just shorthand for ‘ $0 = 0$ ’.

The induction step subtree (2):

$$\begin{array}{c}
\frac{\frac{}{\Gamma_0 \vdash 0 \mathbf{N}} 0I \quad \frac{\frac{}{\Gamma_0 \vdash a \mathbf{N}} H}{\Gamma_0 \vdash \mathbf{S}(a) \mathbf{N}} \mathbf{S}TIE}{\Gamma_0 \vdash \mathbf{S}(a) = 0 \mathbf{B}} =TI \quad \frac{}{\Gamma_0 \vdash x \mathbf{N}} H \quad \frac{(3) \quad \frac{}{\Gamma_0 \vdash \mathbf{add}(x, \mathbf{mult}(x, \mathbf{P}(\mathbf{S}(a)))) \mathbf{N}}}{\Gamma_0 \vdash \mathbf{add}(x, \mathbf{mult}(x, \mathbf{P}(\mathbf{S}(a)))) \mathbf{N}} ?TI}{\Gamma_0 \vdash \mathbf{S}(a) = 0 ? 0 : \mathbf{add}(x, \mathbf{mult}(x, \mathbf{P}(\mathbf{S}(a)))) \mathbf{N}} =IE \\
\hline
\Gamma_0 \vdash \mathbf{mult}(x, \mathbf{S}(a)) \mathbf{N}
\end{array}$$

Where $\Gamma_0 = \{x \mathbf{N}, y \mathbf{N}, a \mathbf{N}, \mathbf{mult}(x, a) \mathbf{N}\}$ for brevity.

Subtree (3):

$$\begin{array}{c}
\frac{\frac{}{\Gamma_0 \vdash a \mathbf{N}} H}{\Gamma_0 \vdash \mathbf{P}(\mathbf{S}(a)) = a} \mathbf{P}=I2 \\
\frac{\Gamma_0 \vdash \mathbf{P}(\mathbf{S}(a)) = a}{\Gamma_0 \vdash a = \mathbf{P}(\mathbf{S}(a))} =S \quad \frac{}{\Gamma_0 \vdash \mathbf{mult}(x, a) \mathbf{N}} H \\
\hline
\frac{\Gamma_0 \vdash x \mathbf{N} \quad H \quad \frac{\Gamma_0 \vdash a = \mathbf{P}(\mathbf{S}(a)) \quad \Gamma_0 \vdash \mathbf{mult}(x, a) \mathbf{N}}{\Gamma_0 \vdash \mathbf{mult}(x, \mathbf{P}(\mathbf{S}(a))) \mathbf{N}} =E}{\Gamma_0 \vdash \mathbf{add}(x, \mathbf{mult}(x, \mathbf{P}(\mathbf{S}(a)))) \mathbf{N}} \text{Theorem A.12}
\end{array}$$

Where $\Gamma_0 = \{x \text{ N}, y \text{ N}, a \text{ N}, \text{mult}(x, a) \text{ N}\}$ for brevity. □

A.6 Typing rule derivations

We begin by deriving the propositional logic typing rules from Table 3.

Theorem A.16. *From the base rules of BGA, we can derive the bidirectional boolean-typing rule for negation:*

$$\frac{p \text{ B}}{\neg p \text{ B}} \neg TIE$$

Proof. In the forward direction, assume that we have already proven ‘ $p \text{ B}$.’ Recall that this is shorthand for the classical LEM, ‘ $p \vee \neg p$,’ and we wish to prove ‘ $\neg p \text{ B}$,’ which is shorthand for ‘ $\neg p \vee \neg \neg p$.’ In the case that ‘ p ’ holds, we can derive ‘ $\neg p \vee \neg \neg p$ ’ by using $\neg \neg IE$ to introduce double negation and then $\vee I2$ to introduce ‘ $\vee$.’ Conversely, if ‘ $\neg p$ ’ holds, we can use $\vee I1$ to introduce ‘ $\neg p \vee \neg \neg p$ ’ directly. Since the judgment is derivable from both disjuncts, we can therefore use standard ‘ $\vee$ ’ elimination to derive ‘ $\neg p \text{ B}$.’ The proof in the reverse direction works largely the same way, using double negation elimination in place of introduction. □

Theorem A.17. *From the base rules of BGA, we can derive disjunction type introduction:*

$$\frac{p \text{ B} \quad q \text{ B}}{p \vee q \text{ B}} \vee TI$$

Proof. Suppose we have already proven ‘ $p \text{ B}$ ’ and ‘ $q \text{ B}$.’ As above, we can perform ‘ $\vee$ ’ elimination on ‘ $p \text{ B}$.’ In the case that ‘ p ’ holds, we can immediately derive ‘ $p \vee q$ ’ and therefore ‘ $(p \vee q) \vee \neg(p \vee q)$.’ In the case that ‘ $\neg p$ ’ holds, we must also perform ‘ $\vee$ ’ elimination on ‘ $q \text{ B}$.’ Once again, in the case that ‘ q ’ holds we are immediately done. Otherwise, we have that both ‘ $\neg p$ ’ and ‘ $\neg q$,’ which allows us to use the negative-case introduction rule $\vee I3$ to derive ‘ $\neg(p \vee q)$.’ In any case, we have that ‘ $p \vee q \text{ B}$.’ □

Theorem A.18. *From the base rules of BGA, we can derive disjunction type elimination:*

$$\frac{p \vee q \text{ B}}{(p \text{ B}) \vee (q \text{ B})} \vee TE$$

Proof. The derivation is again through case analysis of ‘ $p \vee q \text{ B}$.’ In the case that ‘ $p \vee q$ ’ holds, we invoke a second ‘ $\vee$ ’ elimination. If ‘ p ,’ then ‘ $p \vee \neg p$ ’ and therefore ‘ $(p \text{ B}) \vee (q \text{ B})$.’ In the same way, we can get from ‘ q ’ to ‘ $(p \text{ B}) \vee (q \text{ B})$.’ In the case that ‘ $\neg(p \vee q)$,’ we can use either negative-case ‘ $\vee$ ’ elimination rule. Supposing we use $\vee E2$, we then get ‘ $\neg p$ ’ which allows us to derive ‘ $p \text{ B}$ ’ and therefore ‘ $(p \text{ B}) \vee (q \text{ B})$.’ In any case, we can derive ‘ $(p \text{ B}) \vee (q \text{ B})$.’ □

Theorem A.19. *From the base rules of BGA, we can derive conjunction type introduction:*

$$\frac{p \text{ B} \quad q \text{ B}}{p \wedge q \text{ B}} \wedge TI$$

Proof. Recall that our ‘ $\wedge$ ’ rule is shorthand for ‘ $\neg(\neg p \vee \neg q)$.’ Starting with ‘ $\vee$ ’ elimination on ‘ $p \text{ B}$,’ we see that the conclusion is almost immediate from ‘ $\neg p$,’ using double negation introduction to derive ‘ $\neg \neg(\neg p \vee \neg q)$ ’ and therefore ‘ $(\neg p \vee \neg q) \text{ B}$.’ Otherwise, if ‘ p ’ holds, we also perform case analysis on ‘ $q \text{ B}$.’ Again, if ‘ $\neg q$,’ then the conclusion is immediate. Otherwise, we have ‘ p ’ and ‘ q .’ Applying double negation introduction to both of these gives us ‘ $\neg(\neg p)$ ’ and ‘ $\neg(\neg q)$,’ which allows us to use the negative-case ‘ $\vee$ ’ introduction rule to derive ‘ $\neg(\neg p \vee \neg q)$ ’ and therefore ‘ $(\neg p \vee \neg q) \text{ B}$.’ □

Theorem A.20. *From the base rules of BGA, we can derive conjunction type elimination:*

$$\frac{p \wedge q \ B}{(p \ B) \vee (q \ B)} \wedge TE$$

Proof. Assume that we have proven ‘ $p \wedge q \ B$.’ We wish to show that ‘ $(p \ B) \vee (q \ B)$.’ In the case that ‘ $p \wedge q$ ’ holds, we have that ‘ $\neg(\neg p \vee \neg q)$.’ We can invoke negative-case ‘ $\vee$ ’ elimination to get ‘ $\neg p$,’ and therefore ‘ $p \ B$ ’ and ‘ $(p \ B) \vee (q \ B)$.’ In the other case, we have ‘ $\neg\neg(\neg p \vee \neg q)$ ’ which is equivalent to ‘ $\neg p \vee \neg q$ ’ by double negation elimination. We can then perform another case analysis. If ‘ $\neg p$ ’ we have ‘ $p \ B$,’ and if ‘ $\neg q$ ’ we have ‘ $q \ B$.’ In any case we can derive that ‘ $(p \ B) \vee (q \ B)$.’ $\square$

Theorem A.21. *From the base rules of BGA, we can derive implication type introduction:*

$$\frac{p \ B \quad q \ B}{p \rightarrow q \ B} \rightarrow TI$$

Proof. Since ‘ $\rightarrow$ ’ is shorthand for ‘ $\neg p \vee q$,’ this proof is trivial using our above derived rules. Using $\neg TIE$, we can obtain ‘ $\neg p \ B$ ’ which allows us to apply $\vee TI$ to conclude ‘ $\neg p \vee q \ B$ ’ from ‘ $\neg p \ B$ ’ and ‘ $q \ B$.’ $\square$

Theorem A.22. *From the base rules of BGA, we can derive implication type elimination:*

$$\frac{p \rightarrow q \ B}{(p \ B) \vee (q \ B)} \rightarrow TE$$

Proof. Again, unfolding the shorthand for material implication, we derive ‘ $(\neg p \ B) \vee (q \ B)$ ’ from ‘ $p \rightarrow q \ B$ ’ using $\vee TE$. Then we perform case analysis, applying $\neg TIE$ if ‘ $\neg p \ B$ ’ to obtain ‘ $p \ B$.’ In either case we can obtain ‘ $(p \ B) \vee (q \ B)$.’ $\square$

Theorem A.23. *From the base rules of BGA, we can derive biconditional type introduction:*

$$\frac{p \ B \quad q \ B}{p \leftrightarrow q \ B} \leftrightarrow TI$$

Proof. The derivation is through two applications of $\rightarrow TI$ to obtain ‘ $p \rightarrow q \ B$ ’ and ‘ $q \rightarrow p \ B$,’ followed by an application of $\wedge TI$ to obtain ‘ $(p \rightarrow q) \wedge (q \rightarrow p) \ B$.’ $\square$

Theorem A.24. *From the base rules of BGA, we can derive biconditional type elimination:*

$$\frac{p \leftrightarrow q \ B}{p \ B} \leftrightarrow TE1 \quad \frac{p \leftrightarrow q \ B}{q \ B} \leftrightarrow TE2$$

Proof. The elimination forms are stronger than the other typing elimination forms, in the sense that we can derive both ‘ $p \ B$ ’ and ‘ $q \ B$ ’ from ‘ $p \leftrightarrow q \ B$,’ so we cannot use the prior derived rules above in their proofs.

We start with the rule $\leftrightarrow TE1$, where we wish to derive ‘ $p \ B$.’ Unfolding all the shorthands, we get that ‘ $p \leftrightarrow q$ ’ is equivalent to ‘ $\neg(\neg(\neg p \vee q) \vee \neg(\neg q \vee p))$.’ We can perform case analysis on the judgment that this is a boolean. In the case that ‘ $\neg(\neg(\neg p \vee q) \vee \neg(\neg q \vee p))$,’ we can use negative-case ‘ $\vee$ ’ elimination to get the negation of either side of the ‘ $\vee$ ’ statement.

We first examine ‘ $\neg\neg(\neg p \vee q)$,’ which from double negation elimination is equivalent to ‘ $\neg p \vee q$.’ In the case that ‘ $\neg p$,’ we get ‘ $p \ B$ ’ from $\neg TIE$. Otherwise, we have ‘ q ’ and we must examine the other side of the ‘ $\vee$ ’ statement, which is ‘ $\neg q \vee p$ ’ after double-negation elimination. We can perform another case analysis on this. If ‘ $\neg q$ ’ holds, we have a contradiction since we already have that ‘ q ’ holds, and therefore we can derive anything using $\neg E$. Otherwise we have that ‘ p ’ holds, so ‘ $p \ B$.’

Next, we must consider the other case of the boolean judgment, which after double-negation elimination is ' $\neg(\neg p \vee q) \vee \neg(\neg q \vee p)$.' We perform a case elimination on this. If the left-hand disjunct holds, we can use negative-case ' $\vee$ ' elimination to get ' $\neg\neg p$,' which gives us ' p ' and therefore ' $p \text{ B.}$ ' If the right-hand disjunct holds, we can use negative-case ' $\vee$ ' elimination to get ' $\neg\neg q$ ' and therefore ' $q \text{ B.}$ '

In every case, we have derived ' $p \text{ B.}$ ' By symmetry, the proof of $\leftrightarrow \text{TE2}$, which concludes ' $q \text{ B.}$ ' is equivalent. $\square$

Next, we prove the natural number typing rules.

Theorem A.25. *From the base rules of BGA, we can derive the typing rule for successor:*

$$\frac{a \text{ N}}{\text{S}(a) \text{ N}} \text{STIE}$$

Proof. Unfolding the natural number typing judgments, we see this rule is actually equivalent to the bidirectional rule $\text{S}=\text{IE}$. $\square$

Theorem A.26. *From the base rules of BGA, we can derive the conditional-evaluation type-introduction rule:*

$$\frac{c \text{ B} \quad a \text{ N} \quad b \text{ N}}{c ? a : b \text{ N}} ?\text{TI}$$

Proof. This follows by case analysis on ' $c \text{ B.}$ ' In the case that ' c ' holds, we can use $?I1$ along with the judgment that ' $a \text{ N}$ ' to get ' $(c ? a : b) = a$.' We can then use $= E$ in the judgment ' $a \text{ N}$ ' to get ' $c ? a : b \text{ N.}$ ' Similarly, if ' $\neg c$ ' holds, we use the other conditional elimination rule to get ' $(c ? a : b) = b$ ' and equality substitution to get ' $c ? a : b \text{ N.}$ ' In any case we have ' $c ? a : b \text{ N.}$ ' $\square$

A.7 Proof that BGA's operational semantics is single-valued

Theorem A.27. *BGA's reduction rules are deterministic: if ' $t \Downarrow v_1$ ' and ' $t \Downarrow v_2$,' then ' $v_1 = v_2$.'*

Proof. By induction on the derivation ' $t \Downarrow v_1$.' In the case of the rule ' $0 \Downarrow 0$,' the conclusion is immediate because no other rule applies to the term 0. Similarly for variable evaluation, our assignment is fixed so it can only have one map for each variable. Next, we consider the rule ' $\text{S}(a) \Downarrow n + 1$.' Suppose ' $\text{S}(a) \Downarrow v_2$ ' for some term ' v_2 .' The reduction must have come from the same rule because no other rule applies to terms of the form ' $\text{S}(a)$.' By the inductive hypothesis, ' a ' always reduces to n , so applying this rule can only result in $n + 1$. Thus, $v_2 = n + 1$ and the condition is satisfied. The proof is analogous for the rule for ' $\text{P}(a)$.'

Next, we consider the positive-case rule for conditional expressions. Suppose ' $c ? a : b \Downarrow n$.' From the premises, we have ' $c \Downarrow 1$ ' and ' $a \Downarrow n$.' From the inductive hypothesis, ' c ' always evaluates to 1 so we can only apply the positive-case rule. Then, since the evaluation of ' a ' is deterministic, so must be the evaluation of ' $c ? a : b$.' The proof is analogous for the negative-case rule.

The rules for negation work similarly. If ' $\neg p \Downarrow 0$,' we have from induction that ' p ' always evaluates to 1, so we can only ever apply the positive-case rule and get 0. The same reasoning applies to the negative-case rule.

Next, we consider the disjunction rules. First, suppose we have ' $p \vee q \Downarrow 1$ ' because ' $p \Downarrow 1$.' By induction, ' p ' always evaluates to 1 so we can never apply the negative-case rule, which requires that ' p ' evaluate to 0. Thus, the only disjunction rules that apply are the ones that evaluate to 1, so ' $p \vee q$ ' always evaluates to 1. The same reasoning applies to the other positive-case rule. If we instead have ' $p \vee q \Downarrow 0$ ' because both ' p ' and ' q ' evaluate to 0, we have by induction that we can never apply either of the positive-case rules. Thus, ' $p \vee q$ ' always evaluates to 0.

The equality rules are equally straightforward. In the positive case, suppose we have ‘ $a = b \Downarrow 1$.’ By induction, ‘ a ’ and ‘ b ’ are deterministic, so they will always evaluate to the same thing. Thus, we can never apply the negative-case rule and ‘ $a = b$ ’ only ever evaluates to 1. On the other hand, in the negative case, ‘ a ’ and ‘ b ’ will always evaluate to different things so we can only ever apply the negative rule.

Finally, we consider invocation of definitions. Suppose ‘ $\mathbf{d}_i(\vec{a}) \Downarrow m$ ’ because each ‘ a_j ’ evaluates to n_j and the definition body ‘ $D_i(\vec{n}) \Downarrow m$ ’. By induction, the parameters will always evaluate to ‘ $\vec{n}$ ’ and the function body applied to $\vec{n}$ will always evaluate to m , so the function invocation always evaluates to m . This completes the induction, demonstrating that evaluation is always deterministic. $\square$

A.8 Proofs that BGA’s inference rules are truth preserving

For completeness and reference, we include truth-preservation proofs for the inference rules of BGA (Table 2). These proofs are slightly more concise than, but in general closely follow, the corresponding mechanically-verified proofs in Isabelle/HOL discussed in Section 5.

A.8.1 Recursive Definitions

Theorem A.28. *Inference rule $\equiv IE$ preserves truth, i.e., if ‘ $s(\vec{x}) \equiv d(\vec{x})$,’ then the judgment ‘ $\Gamma \vdash p\langle d(\vec{a}), \dots \rangle$ ’ is true under a definition list D if and only if the judgment ‘ $\Gamma \vdash p\langle s(\vec{a}), \dots \rangle$ ’ is true under D .*

Proof. We prove only the forward direction, as the backward direction follows symmetrically.

Let A be an assignment that satisfies all the hypotheses in Γ , and let $\mathbf{d}_i$ be a new definition such that ‘ $\mathbf{d}_i(v) \equiv p\langle v, \dots \rangle$ ’. By the assumption that ‘ $\Gamma \vdash p\langle d(\vec{a}), \dots \rangle$ ’ is true, we know that ‘ $p\langle d(\vec{a}), \dots \rangle \Downarrow 1$ ’, which becomes ‘ $\mathbf{d}_i(d(\vec{a})) \Downarrow 1$ ’.

For ‘ $\mathbf{d}_i(d(\vec{a})) \Downarrow 1$ ’ to hold, there must exist a sequence of reduction rules that ends with ‘ $\mathbf{d}_i(d(\vec{a})) \Downarrow 1$ ’. Since there is only one reduction rule with the suitable conclusion, the last reduction in the sequence has to be

$$\frac{d(\vec{a}) \Downarrow n \quad p\langle \underline{n}, \dots \rangle \Downarrow 1}{\mathbf{d}_i(d(\vec{a})) \Downarrow 1}$$

Thus, we have ‘ $d(\vec{a}) \Downarrow n$ ’ and ‘ $p\langle \underline{n}, \dots \rangle \Downarrow 1$ ’. By the definition of s , it follows that ‘ $s(\vec{a}) \Downarrow n$ ’. Finally, by applying the same reduction rule on ‘ $s(\vec{a}) \Downarrow n$ ’ and ‘ $p\langle \underline{n}, \dots \rangle \Downarrow 1$ ’, we obtain ‘ $\mathbf{d}_i(s(\vec{a})) \Downarrow 1$ ’, i.e., ‘ $p\langle s(\vec{a}), \dots \rangle \Downarrow 1$ ’. $\square$

A.8.2 Equality

Lemma A.29. *For any terms ‘ a ’ and ‘ b ,’ under any assignment A , and a definition list D*

$$\text{if ‘} a = b \Downarrow 1 \text{’ and ‘} p\langle a, \dots \rangle \Downarrow 1 \text{,’ then ‘} p\langle b, \dots \rangle \Downarrow 1 \text{.’}$$

Proof. Let $\mathbf{d}_i$ be a new definition such that ‘ $\mathbf{d}_i(a) \equiv p\langle a, \dots \rangle$ ’. Then the assumption ‘ $p\langle a, \dots \rangle \Downarrow 1$ ’ becomes ‘ $\mathbf{d}_i(a) \Downarrow 1$ ’.

For ‘ $a = b \Downarrow 1$ ’ and ‘ $\mathbf{d}_i(a) \Downarrow 1$ ’ to hold, there must exist two sequences of reduction rules that end with ‘ $a = b \Downarrow 1$ ’ and ‘ $\mathbf{d}_i(a) \Downarrow 1$ ’ as their conclusions. Based on the reduction rules available, this is possible if and only if the sequences end with the following rules, respectively:

$$\frac{a \Downarrow n \quad b \Downarrow n}{a = b \Downarrow 1} \quad \frac{a \Downarrow m \quad p\langle \underline{m}, \dots \rangle \Downarrow 1}{\mathbf{d}_i(a) \Downarrow 1}$$

Therefore, ‘ $a \Downarrow n$ ’, ‘ $b \Downarrow n$ ’, ‘ $a \Downarrow m$ ’, and ‘ $p\langle \underline{m}, \dots \rangle \Downarrow 1$ ’ hold. Since BGA’s operational semantics is single-valued (Theorem A.27), it follows that $n = m$, and hence ‘ $b \Downarrow m$ ’.

Finally, we can apply the second reduction rule on the term ‘ b ’ instead of ‘ a ’, and infer that ‘ $\mathbf{d}_i(b) \Downarrow 1$ ’, i.e., ‘ $p\langle b, \dots \rangle \Downarrow 1$ ’. $\square$

Theorem A.30. *Inference rule $=S$ preserves truth, i.e., if the judgment ‘ $\Gamma \vdash a = b$ ’ is true under a definition list D , then the judgment ‘ $\Gamma \vdash b = a$ ’ is also true under D .*

Proof. Let A be an assignment that satisfies all the hypotheses in Γ . Then, by assumption that ‘ $\Gamma \vdash a = b$ ’ is true, we have ‘ $a = b \Downarrow 1$.’

That means there is a sequence of reduction rules that ends with ‘ $a = b \Downarrow 1$ ’ as its final conclusion. Based on the available reduction rules, that is possible if and only if the last reduction rule used was

$$\frac{a \Downarrow n \quad b \Downarrow n}{a = b \Downarrow 1}.$$

Thus, the sequence of reduction rules contains conclusions ‘ $a \Downarrow n$ ’ and ‘ $b \Downarrow n$,’ for some n . Then we can apply the same rule with ‘ a ’ and ‘ b ’ reversed, and conclude that ‘ $b = a \Downarrow 1$.’ $\square$

Theorem A.31. *Inference rule $=E$ preserves truth, i.e., if the judgments ‘ $\Gamma \vdash a = b$ ’ and ‘ $\Gamma \vdash p\langle a, \dots \rangle$ ’ are true under a definition list D , then the judgment ‘ $\Gamma \vdash p\langle b, \dots \rangle$ ’ is also true under D .*

Proof. Let A be an assignment that satisfies all the hypotheses in Γ . Then, by the assumptions that ‘ $\Gamma \vdash a = b$ ’ and ‘ $\Gamma \vdash p\langle a, \dots \rangle$ ’ are true, we have ‘ $a = b \Downarrow 1$ ’ and ‘ $p\langle a, \dots \rangle \Downarrow 1$.’ Then, using Lemma A.29 we derive the conclusion ‘ $p\langle b, \dots \rangle \Downarrow 1$.’ $\square$

A.8.3 Propositional logic

Lemma A.32. *For any term ‘ p ,’ under any assignment A , and a definition list D*

$$\text{‘}\neg p \Downarrow 1\text{’ if and only if ‘}p \Downarrow 0\text{.’}$$

Proof. Forward direction: Based on the assumption that ‘ $\neg p \Downarrow 1$,’ we can infer that there exists a sequence of reduction steps that ends with ‘ $\neg p \Downarrow 1$ ’ as its final conclusion. Then, by case analysis on the available reduction rules, we conclude that the last reduction in the sequence has to be

$$\frac{p \Downarrow 0}{\neg p \Downarrow 1}$$

Thus, the sequence also contains ‘ $p \Downarrow 0$ ’ as one of the conclusions, which implies that ‘ $p \Downarrow 0$ ’ holds.

Backward direction: It follows by applying the appropriate reduction rule. $\square$

Lemma A.33. *For any term ‘ p ,’ under any assignment A , and a definition list D*

$$\text{‘}\neg p \Downarrow 0\text{’ if and only if ‘}p \Downarrow 1\text{.’}$$

Proof. The statement follows from reasoning similar to that in the proof of Lemma A.33, but using a different reduction rule, specifically:

$$\frac{p \Downarrow 1}{\neg p \Downarrow 0}$$

$\square$

Theorem A.34. *Inference rule $\neg\neg IE$ preserves truth, i.e., the judgment ‘ $\Gamma \vdash p$ ’ is true under a definition list D if and only if the judgment ‘ $\Gamma \vdash \neg\neg p$ ’ is true under D .*

Proof. Forward direction: Let A be an assignment satisfying all the hypotheses in Γ . Then, based on the assumption that ' $p \Downarrow 1$,' we have ' $\Gamma \vdash \neg\neg p$.' Applying the reduction rules for negation twice, we get that ' $\neg\neg p \Downarrow 1$.'

Backward direction: Let A be an assignment satisfying all the hypotheses in Γ . Based on the assumption that ' $\Gamma \vdash \neg\neg p$ ' is true, we have ' $\neg\neg p \Downarrow 1$.' Then, by applying Lemma A.32 and Lemma A.33 respectively, we get that ' $p \Downarrow 1$ ' holds. $\square$

Theorem A.35. *Inference rule $\neg E$ preserves truth, i.e., if the judgments ' $\Gamma \vdash p$ ' and ' $\Gamma \vdash \neg p$ ' are true under a definition list D , then the judgment ' $\Gamma \vdash q$ ' is also true under D .*

Proof. Let A be an assignment that satisfies all the hypotheses in Γ . Based on the assumptions that ' $\Gamma \vdash p$ ' and ' $\Gamma \vdash \neg p$ ' are both true, we conclude that ' $p \Downarrow 1$ ' and ' $\neg p \Downarrow 1$.' However, ' $p \Downarrow 1$ ' implies ' $\neg p \Downarrow 0$,' and since BGA's operational semantics is single-valued (Theorem A.27), this leads to a contradiction. Therefore, ' $q \Downarrow 1$ ' holds vacuously. $\square$

Theorem A.36. *Inference rule $\vee I1$ preserves truth, i.e., if the judgment ' $\Gamma \vdash p$ ' is true under a definition list D , then the judgment ' $\Gamma \vdash p \vee q$ ' is also true under D .*

Proof. Let A be an assignment that satisfies all the hypotheses in Γ . Based on the assumption that ' $\Gamma \vdash p$ ' is true, we conclude that ' $p \Downarrow 1$.' Then, using a reduction rule, we induce ' $p \vee q \Downarrow 1$.' $\square$

Theorem A.37. *Inference rule $\vee I2$ preserves truth, i.e., if the judgment ' $\Gamma \vdash q$ ' is true under a definition list D , then the judgment ' $\Gamma \vdash p \vee q$ ' is also true under D .*

Proof. The statement follows from a proof similar to that of Theorem A.36, differing only in the use of a different reduction rule at the end. $\square$

Theorem A.38. *Inference rule $\vee I3$ preserves truth, i.e., if the judgments ' $\Gamma \vdash \neg p$ ' and ' $\Gamma \vdash \neg q$ ' are true under a definition list D , then the judgment ' $\Gamma \vdash \neg(p \vee q)$ ' is also true under D .*

Proof. Let A be an assignment that satisfies all the hypotheses in Γ . Based on the assumptions that ' $\Gamma \vdash \neg p$ ' and ' $\Gamma \vdash \neg q$ ' are true, we conclude that ' $\neg p \Downarrow 1$ ' and ' $\neg q \Downarrow 1$.'

Using Lemma A.32, we get ' $p \Downarrow 0$ ' and ' $q \Downarrow 0$.' Then, by the reduction rules:

$$\frac{p \Downarrow 0 \quad q \Downarrow 0}{p \vee q \Downarrow 0} \quad \frac{p \vee q \Downarrow 0}{\neg(p \vee q) \Downarrow 1}$$

we derive the desired result ' $\neg(p \vee q) \Downarrow 1$.' $\square$

Theorem A.39. *Inference rule $\vee E1$ preserves truth, i.e., if the judgments*

$$' \Gamma \vdash p \vee q ' \quad ' \Gamma, p \vdash r ' \quad ' \Gamma, q \vdash r '$$

are true under a definition list D , then the judgment ' $\Gamma \vdash r$ ' is also true under D .

Proof. Let A be an assignment that satisfies all the hypotheses in Γ . Based on the assumptions that ' $\Gamma \vdash p \vee q$ ' is true, we conclude that ' $p \vee q \Downarrow 1$.' That means there exists a sequence of reduction rules that ends with ' $p \vee q \Downarrow 1$ ' as its final conclusion. By using case analysis on the available reduction rules, we infer that the last reduction in the sequence has to be one of the following:

$$\frac{p \Downarrow 1}{p \vee q \Downarrow 1} \quad \frac{q \Downarrow 1}{p \vee q \Downarrow 1}$$

If the first one holds, then ' $p \Downarrow 1$ ' holds, and using the assumption that ' $\Gamma, p \vdash r$ ' is true, we infer that ' $r \Downarrow 1$ '.

Otherwise, if the second one holds, then ' $q \Downarrow 1$,' and using the assumption that ' $\Gamma, q \vdash r$ ' is true, we get ' $r \Downarrow 1$.' $\square$

Theorem A.40. *Inference rules $\vee E2$ and $\vee E3$ preserve truth, i.e., if the judgment ' $\Gamma \vdash \neg(p \vee q)$ ' is true under a definition list D , then the judgments ' $\Gamma \vdash \neg p$ ' and ' $\Gamma \vdash \neg q$ ' are also true under D .*

Proof. Let A be an assignment that satisfies all the hypotheses in Γ . Based on the assumption that ' $\Gamma \vdash \neg(p \vee q)$ ' is true, we conclude that ' $\neg(p \vee q) \Downarrow 1$.' Using Lemma A.32, we have that ' $p \vee q \Downarrow 0$.'

That means there exists a sequence of reduction rules that ends with ' $p \vee q \Downarrow 0$ ' as its final conclusion. Using case analysis on the available reduction rules, we conclude that the last reduction in the sequence is

$$\frac{p \Downarrow 0 \quad q \Downarrow 0}{p \vee q \Downarrow 0}$$

Thus, the sequence includes ' $p \Downarrow 0$ ' and ' $q \Downarrow 0$ ' as conclusions, and therefore these results hold. Finally, using a reduction rule for negation, we get ' $\neg p \Downarrow 1$ ' and ' $\neg q \Downarrow 1$.' $\square$

A.8.4 Natural numbers

Lemma A.41. *For any terms ' a ' and ' b ,' under any assignment A , and a definition list D*

' $a = b \Downarrow 1$ ' if and only if there exists n such that ' $a \Downarrow n$ ' and ' $b \Downarrow n$.'

Proof. Forward direction: If ' $a = b \Downarrow 1$,' then there exists a sequence of reduction rules that ends with ' $a = b \Downarrow 1$ ' as its final conclusion. Using case analysis on the available reduction rules, we conclude that the last reduction in that sequence is:

$$\frac{a \Downarrow n \quad b \Downarrow n}{a = b \Downarrow 1}$$

for some n . Thus, ' $a \Downarrow n$ ' and ' $b \Downarrow n$ ' holds.

Backward direction: It follows directly by apply the appropriate reduction rule. $\square$

Lemma A.42. *For any terms ' a ' and ' b ,' under any assignment A , and a definition list D*

' $a = b \Downarrow 0$ ' if and only if there exist n and m such that ' $a \Downarrow n$,' ' $b \Downarrow m$,' and $n \neq m$.

Proof. Forward direction: For ' $a = b \Downarrow 0$ ' to hold, there has to be a sequence of reduction rules that ends with ' $a = b \Downarrow 0$ ' as its final conclusion. Using case analysis on the available reduction rules, we conclude that the last reduction in the sequence is:

$$\frac{a \Downarrow n \quad b \Downarrow m \quad n \neq m}{a = b \Downarrow 0}$$

Thus, ' $a \Downarrow n$ ' and ' $b \Downarrow m$ ' also belong to that sequence, for some n and m such that $n \neq m$, and consequently hold.

Backward direction: It follows directly by applying the appropriate reduction rule. $\square$

Lemma A.43. *For any term ' a ,' under any assignment A , and a definition list D*

' $S(a) \Downarrow n + 1$ ' if and only if ' $a \Downarrow n$.'

Proof. Forward direction: If ‘ $S(a) \Downarrow n + 1$ ’ holds, then there exists a sequence of reduction rules that ends with ‘ $S(a) \Downarrow n + 1$ ’ as its final conclusion. Then, using case analysis on the available reduction rules, we conclude that the last reduction in the sequence is

$$\frac{a \Downarrow n}{S(a) \Downarrow n + 1}$$

Thus, the ‘ $a \Downarrow n$ ’ holds.

Backward direction: It follows directly by applying the appropriate reduction rule. $\square$

Lemma A.44. *For any term ‘ a ,’ under any assignment A , and a definition list D*

if ‘ $S(a) \Downarrow n$,’ then $n = m + 1$ for some m .

Proof. If ‘ $S(a) \Downarrow n$ ’ then there exists a sequence of reduction rules that ends with ‘ $S(a) \Downarrow n$ ’ as its final conclusion. Using case analysis on the available reduction rules, it follows that the last reduction rule applied in that sequence is

$$\frac{a \Downarrow m}{S(a) \Downarrow m + 1}$$

Thus, $n = m + 1$ for some m . $\square$

Theorem A.45. *Inference rule 0I preserves truth, i.e., the judgment ‘ $\Gamma \vdash 0 \ N$ ’ is true under any definition list D .*

Proof. Since ‘ $0 \ N$ ’ is just a shorthand for ‘ $0 = 0$,’ we get the result by using the following two reduction rules:

$$\frac{}{0 \Downarrow 0} \quad \frac{0 \Downarrow 0 \quad 0 \Downarrow 0}{0 = 0 \Downarrow 1}$$

$\square$

Theorem A.46. *Inference rule $S=IE$ preserves truth, i.e., the judgment ‘ $\Gamma \vdash a = b$ ’ is true under a definition list D if and only if the judgment ‘ $\Gamma \vdash S(a) = S(b)$ ’ is true under D .*

Proof. Forward direction: Let A be an assignment that satisfies all the hypotheses in Γ . Then, by using the assumption that ‘ $\Gamma \vdash a = b$ ’ is true, we conclude that ‘ $a = b \Downarrow 1$.’ Using Lemma A.41 we also have that ‘ $a \Downarrow n$ ’ and ‘ $b \Downarrow n$ ’ for some n .

Then we get the desired result by applying the following reduction rules:

$$\frac{a \Downarrow n}{S(a) \Downarrow n + 1} \quad \frac{b \Downarrow n}{S(b) \Downarrow n + 1} \quad \frac{S(a) \Downarrow n + 1 \quad S(b) \Downarrow n + 1}{S(a) = S(b) \Downarrow 1}$$

Backward direction: Let A be an assignment that satisfies all the hypotheses in Γ . Based on the assumption that ‘ $\Gamma \vdash S(a) = S(b)$ ’ is true, we conclude that ‘ $S(a) = S(b) \Downarrow 1$.’

Using Lemma A.41, we have that ‘ $S(a) \Downarrow n$ ’ and ‘ $S(b) \Downarrow n$ ’ for some n . Moreover, using Lemma A.44, we have that $n = m + 1$ for some m , therefore, ‘ $S(a) \Downarrow m + 1$ ’ and ‘ $S(b) \Downarrow m + 1$.’ Finally, by Lemma A.43, we infer ‘ $a \Downarrow m$ ’ and ‘ $b \Downarrow m$,’ and we prove the desired result using the reduction rule:

$$\frac{a \Downarrow m \quad b \Downarrow m}{a = b \Downarrow 1}$$

$\square$

Theorem A.47. *Inference rule $\mathbf{S} \neq IE$ preserves truth, i.e., the judgment ' $\Gamma \vdash a \neq b$ ' is true under a definition list D if and only if the judgment ' $\Gamma \vdash \mathbf{S}(a) \neq \mathbf{S}(b)$ ' is true under D .*

Proof. *Forward direction:* Let A be an assignment that satisfies all the hypotheses in Γ . Using the assumption that ' $\Gamma \vdash a \neq b$ ' is true, we conclude that ' $a \neq b \Downarrow 1$.' Since ' $a \neq b$ ' is just a shorthand for ' $\neg(a = b)$,' it follows from Lemma A.32 that ' $a = b \Downarrow 0$.'

Then, using the Lemma A.42, we get that there exists n such that ' $a \Downarrow n$,' ' $b \Downarrow m$,' and $n \neq m$. Finally, we deduce the desired conclusion from Lemma A.32 and the following reduction rules:

$$\frac{a \Downarrow n}{\mathbf{S}(a) \Downarrow n+1} \quad \frac{b \Downarrow m}{\mathbf{S}(b) \Downarrow m+1} \quad \frac{\mathbf{S}(a) \Downarrow n+1 \quad \mathbf{S}(b) \Downarrow m+1 \quad n+1 \neq m+1}{\mathbf{S}(a) = \mathbf{S}(b) \Downarrow 0}$$

Backward direction: Let A be an assignment that satisfies all the hypotheses in Γ . Using the assumption that ' $\Gamma \vdash \mathbf{S}(a) \neq \mathbf{S}(b)$ ' is true, we conclude that ' $\mathbf{S}(a) \neq \mathbf{S}(b) \Downarrow 1$.' Since ' $\mathbf{S}(a) \neq \mathbf{S}(b)$ ' is just a shorthand for ' $\neg(\mathbf{S}(a) = \mathbf{S}(b))$,' it follows from Lemma A.32 that ' $\mathbf{S}(a) = \mathbf{S}(b) \Downarrow 0$.'

Then, by Lemma A.42, we have that ' $\mathbf{S}(a) \Downarrow n$,' ' $\mathbf{S}(b) \Downarrow m$,' and $n \neq m$. Furthermore, Lemma A.44 guarantees the existence of n' and m' such that $n = n' + 1$ and $m = m' + 1$, and by Lemma A.43, it follows that ' $a \Downarrow n'$ ' and ' $b \Downarrow m'$ '.

Finally, we conclude ' $a \neq b \Downarrow 1$ ' by using Lemma A.32 and the reduction rule:

$$\frac{a \Downarrow n' \quad b \Downarrow m' \quad n' \neq m'}{a \neq b \Downarrow 1}$$

□

Theorem A.48. *Inference rule $\mathbf{S} \neq 0I$ preserves truth, i.e., if the judgment ' $\Gamma \vdash a \mathbf{N}$ ' is true under a definition list D , then the judgment ' $\Gamma \vdash \mathbf{S}(a) \neq 0$ ' is also true under D .*

Proof. Let A be an assignment that satisfies all the hypotheses in Γ . Using the assumption that ' $\Gamma \vdash a \mathbf{N}$ ' is true, we conclude that ' $a \mathbf{N} \Downarrow 1$.' Since ' $a \mathbf{N}$ ' is just a shorthand for ' $a = a$,' using Lemma A.41, we have that there exists n such that ' $a \Downarrow n$ '.

Then, we get ' $\mathbf{S}(a) = 0 \Downarrow 0$ ' by using the following reduction rules:

$$\frac{}{0 \Downarrow 0} \quad \frac{a \Downarrow n}{\mathbf{S}(a) \Downarrow n+1} \quad \frac{\mathbf{S}(a) \Downarrow n+1 \quad 0 \Downarrow 0 \quad 0 \neq n+1}{\mathbf{S}(a) = 0 \Downarrow 0}$$

Finally, since ' $\mathbf{S}(a) \neq 0$ ' is a shorthand for ' $\neg(\mathbf{S}(a) = 0)$,' we obtain the desired result by Lemma A.32. □

Theorem A.49. *Inference rule $\mathbf{P} = I2$ preserves truth, i.e., if the judgment ' $\Gamma \vdash a \mathbf{N}$ ' is true under a definition list D , then the judgment ' $\Gamma \vdash \mathbf{P}(\mathbf{S}(a)) = a$ ' is also true under D .*

Proof. Let A be an assignment that satisfies all the hypotheses in Γ . Using the assumption that ' $\Gamma \vdash a \mathbf{N}$ ' is true, we conclude that ' $a \mathbf{N} \Downarrow 1$.' Since ' $a \mathbf{N}$ ' is just a shorthand for ' $a = a$,' using Lemma A.41, we have that there exists n such that ' $a \Downarrow n$ '.

We obtain the end result by applying the following reduction rules:

$$\frac{a \Downarrow n}{\mathbf{S}(a) \Downarrow n+1} \quad \frac{\mathbf{S}(a) \Downarrow n+1}{\mathbf{P}(\mathbf{S}(a)) \Downarrow n} \quad \frac{\mathbf{P}(\mathbf{S}(a)) \Downarrow n \quad a \Downarrow n}{\mathbf{P}(\mathbf{S}(a)) = a \Downarrow 1}$$

□

Theorem A.50. *Inference rule $\mathbf{P} TIE$ preserves truth, i.e., the judgment ' $\Gamma \vdash a \mathbf{N}$ ' is true under a definition list D if and only if the judgment ' $\Gamma \vdash \mathbf{P}(a)$ ' is true under D .*

Proof. Forward direction: Let A be an assignment that satisfies all the hypotheses in Γ . Using the assumption that ' $\Gamma \vdash a \mathbf{N}$ ' is true, we conclude that ' $a \mathbf{N} \Downarrow 1$.' Since ' $a \mathbf{N}$ ' is just a shorthand for ' $a = a$,' by Lemma A.41, we also have that ' $a \Downarrow n$ ' for some n .

We now distinguish two cases: $n = 0$ and $n \neq 0$.

If $n = 0$, we get the desired result by using the following reduction rules:

$$\frac{a \Downarrow 0}{\mathbf{P}(a) \Downarrow 0} \quad \frac{\mathbf{P}(a) \Downarrow 0 \quad \mathbf{P}(a) \Downarrow 0}{\mathbf{P}(a) = \mathbf{P}(a) \Downarrow 1}$$

If $n \neq 0$, then there exists a natural number m such that $n = m + 1$, and we get the desired result by using the following reduction rules:

$$\frac{a \Downarrow m + 1}{\mathbf{P}(a) \Downarrow m} \quad \frac{\mathbf{P}(a) \Downarrow m \quad \mathbf{P}(a) \Downarrow m}{\mathbf{P}(a) = \mathbf{P}(a) \Downarrow 1}$$

Backward direction: Let A be an assignment that satisfies all the hypotheses in Γ . Using the assumption that ' $\Gamma \vdash \mathbf{P}(a) \mathbf{N}$ ' is true, we conclude that ' $\mathbf{P}(a) \mathbf{N} \Downarrow 1$.' Since ' $\mathbf{P}(a) \mathbf{N} \Downarrow 1$ ' is just a shorthand for ' $\mathbf{P}(a) = \mathbf{P}(a) \Downarrow 1$,' using Lemma A.41, we also have that ' $\mathbf{P}(a) \Downarrow n$ ' for some n .

That means there exists a sequence of reduction rules that ends with ' $\mathbf{P}(a) \Downarrow n$ ' as its final conclusion. Using case analysis on the available reduction rules, we infer that the last reduction in that sequence is one of the following:

$$\frac{a \Downarrow 0}{\mathbf{P}(a) \Downarrow 0} \quad \frac{a \Downarrow n + 1}{\mathbf{P}(a) \Downarrow n}$$

In any case, we get that ' $a \Downarrow m$ ' for some m . Then the end result follows from the reduction rule:

$$\frac{a \Downarrow m \quad a \Downarrow m}{a = a \Downarrow 1}$$

□

Theorem A.51. *Inference rule ?I1 preserves truth, i.e., if the judgments ' $\Gamma \vdash c$ ' and ' $\Gamma \vdash a \mathbf{N}$ ' are true under a definition list ' D ,' then the judgment ' $\Gamma \vdash (c ? a : b) = a$ ' is also true under D .*

Proof. Let A be an assignment that satisfies all the hypotheses in Γ . From the assumptions that ' $\Gamma \vdash c$ ' and ' $\Gamma \vdash a \mathbf{N}$ ' are both true, we conclude that ' $c \Downarrow 1$,' and ' $a \mathbf{N} \Downarrow 1$.' Since ' $a \mathbf{N}$ ' is just a shorthand for ' $a = a$,' by using Lemma A.41, we also have that ' $a \Downarrow n$ ' for some n .

The end result follows by applying the reduction rules:

$$\frac{c \Downarrow 1 \quad a \Downarrow n}{c ? a : b \Downarrow n} \quad \frac{c ? a : b \Downarrow n \quad a \Downarrow n}{(c ? a : b) = a \Downarrow 1}$$

□

Theorem A.52. *Inference rule ?I2 preserves truth, i.e., if the judgments ' $\Gamma \vdash \neg c$ ' and ' $\Gamma \vdash b \mathbf{N}$ ' are true under a definition list D , then the judgment ' $\Gamma \vdash (c ? a : b) = b$ ' is also true under D .*

Proof. Let A be an assignment that satisfies all the hypotheses in Γ . From the assumptions that ' $\Gamma \vdash \neg c$ ' and ' $\Gamma \vdash b \mathbf{N}$ ' are both true, we conclude that ' $\neg c \Downarrow 1$,' and ' $b \mathbf{N} \Downarrow 1$.' Using Lemma A.32, we have that ' $c \Downarrow 0$.' Moreover, since ' $b \mathbf{N}$ ' is just a shorthand for ' $b = b$,' by using Lemma A.41, we also have that ' $b \Downarrow n$ ' for some n .

The end result follows by applying the reduction rules:

$$\frac{c \Downarrow 0 \quad b \Downarrow n}{c ? a : b \Downarrow n} \quad \frac{c ? a : b \Downarrow n \quad b \Downarrow n}{(c ? a : b) = b \Downarrow 1}$$

□

Theorem A.53. *Inference rule Ind preserves truth, i.e., if the judgments*

$$\langle \Gamma \vdash p\langle 0, \dots \rangle \rangle \quad \langle \Gamma, x \mathbf{N}, p\langle x, \dots \rangle \vdash p\langle \mathbf{S}(x), \dots \rangle \rangle \quad \langle \Gamma \vdash a \mathbf{N} \rangle$$

are true under a definition list D , and neither of the hypotheses in Γ contains x as a free variable, then the judgment $\langle \Gamma \vdash p\langle a, \dots \rangle \rangle$ is also true under D .

Proof. Let A be an assignment that satisfies all the hypotheses in Γ .

Using the assumptions that $\langle \Gamma \vdash p\langle 0, \dots \rangle \rangle$ and $\langle \Gamma, x \mathbf{N}, p\langle x, \dots \rangle \vdash p\langle \mathbf{S}(x), \dots \rangle \rangle$ are true, we have that

$$\langle p\langle 0, \dots \rangle \Downarrow 1 \rangle \tag{1}$$

$$\text{if } \langle x \mathbf{N} \Downarrow 1 \rangle \text{ and } \langle p\langle x, \dots \rangle \Downarrow 1 \rangle, \text{ then } \langle p\langle \mathbf{S}(x), \dots \rangle \Downarrow 1 \rangle. \tag{2}$$

for each term ' x ' that is not free in all of the hypotheses in Γ .

Now we would like to prove the following:

$$\text{if } \langle a \Downarrow n \rangle \text{ for some term } \langle a \rangle \text{ and natural number } n, \text{ then } \langle p\langle a, \dots \rangle \Downarrow 1 \rangle \tag{3}$$

We do that by induction on n .

Base case: Here we assume that $\langle a \Downarrow 0 \rangle$ and aim to prove that $\langle p\langle a, \dots \rangle \Downarrow 1 \rangle$.

By using the reduction rule:

$$\frac{a \Downarrow 0 \quad 0 \Downarrow 0}{a = 0 \Downarrow 1}$$

we get $\langle a = 0 \Downarrow 1 \rangle$. Then, by using (1) and Lemma A.29, we get the desired conclusion, i.e., $\langle p\langle a, \dots \rangle \Downarrow 1 \rangle$.

Induction step: Here we assume that $\langle a \Downarrow n + 1 \rangle$ and aim to show that $\langle p\langle a, \dots \rangle \Downarrow 1 \rangle$ using the induction hypothesis.

Using the reduction rule

$$\frac{a \Downarrow n + 1}{\mathbf{P}(a) \Downarrow n}$$

we get that $\langle \mathbf{P}(a) \Downarrow n \rangle$. Hence, we can use the induction hypothesis on the term ' $\mathbf{P}(a)$,' and we have $\langle p\langle \mathbf{P}(a), \dots \rangle \Downarrow 1 \rangle$. We also have $\langle \mathbf{P}(a) \mathbf{N} \Downarrow 1 \rangle$ by using the reduction rule

$$\frac{\mathbf{P}(a) \Downarrow n \quad \mathbf{P}(a) \Downarrow n}{\mathbf{P}(a) = \mathbf{P}(a) \Downarrow 1}$$

Thus, we can apply (2) to conclude that $\langle p\langle \mathbf{S}(\mathbf{P}(a)), \dots \rangle \Downarrow 1 \rangle$. Moreover, it also holds $\langle \mathbf{S}(\mathbf{P}(a)) = a \Downarrow 1 \rangle$ by

$$\frac{\mathbf{P}(a) \Downarrow n}{\mathbf{S}(\mathbf{P}(a)) \Downarrow n + 1} \quad \frac{\mathbf{S}(\mathbf{P}(a)) \Downarrow n + 1 \quad a \Downarrow n + 1}{\mathbf{S}(\mathbf{P}(a)) = a \Downarrow 1}$$

Finally, using Lemma A.29 on $\langle p\langle \mathbf{S}(\mathbf{P}(a)), \dots \rangle \Downarrow 1 \rangle$ and $\langle \mathbf{S}(\mathbf{P}(a)) = a \Downarrow 1 \rangle$, we obtain that $\langle p\langle a, \dots \rangle \Downarrow 1 \rangle$.

Therefore, for each n and ' a ' for which $\langle a \Downarrow n \rangle$ holds, $\langle p\langle a, \dots \rangle \Downarrow 1 \rangle$ also holds.

To finish the proof, we use the final assumption of the theorem: that $\langle \Gamma \vdash a \mathbf{N} \rangle$ is true. This yields that $\langle a \mathbf{N} \Downarrow 1 \rangle$ under A and D . Since $\langle a \mathbf{N} \Downarrow 1 \rangle$ is a shorthand for $\langle a = a \Downarrow 1 \rangle$, using Lemma A.41, we obtain that $\langle a \Downarrow n \rangle$ for some n . Finally, using (3) we conclude that $\langle p\langle a, \dots \rangle \Downarrow 1 \rangle$ holds. □

A.8.5 Structural rules

Theorem A.54. *Inference rule H preserves truth, i.e., the judgment $\Gamma, p \vdash p$ is true under any definition list D .*

Proof. Let A be an assignment that satisfies all the hypotheses in the list Γ, p . Specifically, ' $p \Downarrow 1$ ' under A and D , thus, the judgment ' $\Gamma, p \vdash p$ ' is true. $\square$

Theorem A.55. *Inference rule W preserves truth, i.e., if the judgment $\Gamma \vdash q$ is true under a definition list D , then the judgment $\Gamma, p \vdash q$ is also true under D .*

Proof. Let A be an assignment that satisfies all the hypotheses in the list Γ, p . Using the assumption that ' $\Gamma \vdash q$ ' is true, we conclude that ' $q \Downarrow 1$ ' holds. Thus, for any assignment that satisfies all the hypotheses in Γ and ' p ,' the conclusion ' q ' is also satisfied, therefore, the judgment ' $\Gamma, p \vdash q$ ' is true. $\square$

Theorem A.56. *Inference rule C preserves truth, i.e., if the judgment $\Gamma, p, p \vdash q$ is true under a definition list D , then the judgment $\Gamma, p \vdash q$ is also true under D .*

Proof. Let A be an assignment that satisfies all the hypotheses in the list Γ, p, p . That is, ' $p \Downarrow 1$ ' and ' $H \Downarrow 1$ ' under A and D , for all hypotheses H in Γ .

Such A also satisfies all the hypotheses in the list Γ, p . Using the assumption that ' $\Gamma, p, p \vdash q$ ' is true, we conclude that ' $q \Downarrow 1$.' Therefore, the judgment ' $\Gamma, p \vdash q$ ' is also true under D . $\square$

Theorem A.57. *Inference rule P preserves truth, i.e., if the judgment $\Gamma, p, q, \Delta \vdash r$ is true under a definition list D , then the judgment $\Gamma, q, p, \Delta \vdash r$ is also true under D .*

Proof. Let A be an assignment that satisfies all the hypotheses in the list ' Γ, q, p, Δ .' That is, for each hypothesis H in Γ , and for each hypothesis H' in Δ , the reductions ' $H \Downarrow 1$,' ' $q \Downarrow 1$,' ' $p \Downarrow 1$ ' and ' $H' \Downarrow 1$ ' hold under A and D .

However, A also satisfies all the hypotheses in the list Γ, p, q, Δ . Then, using the assumption that ' $\Gamma, p, q, \Delta \vdash r$ ' is true, we conclude that ' $r \Downarrow 1$.' $\square$